



中国科学院大学
University of Chinese Academy of Sciences

Deep Learning

Transformer

Xinfeng Zhang (张新峰)

School of Computer Science and Technology

University of Chinese Academy of Sciences

Email: xfzhang@ucas.ac.cn



计算机科学与技术学院

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY



提纲

- 注意力机制
- Transformer
- GPT系列模型
- BERT系列模型
- Swin Transformer
- Transformer的主要应用
- 中英文术语对照



1

注意力机制

人类注意力机制



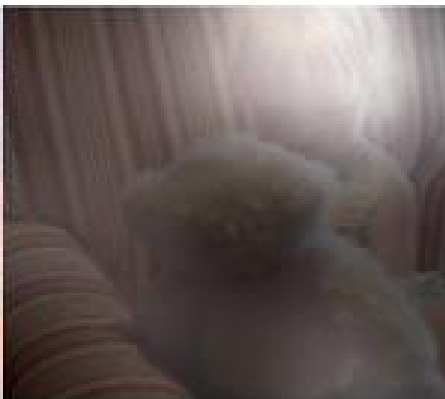
A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



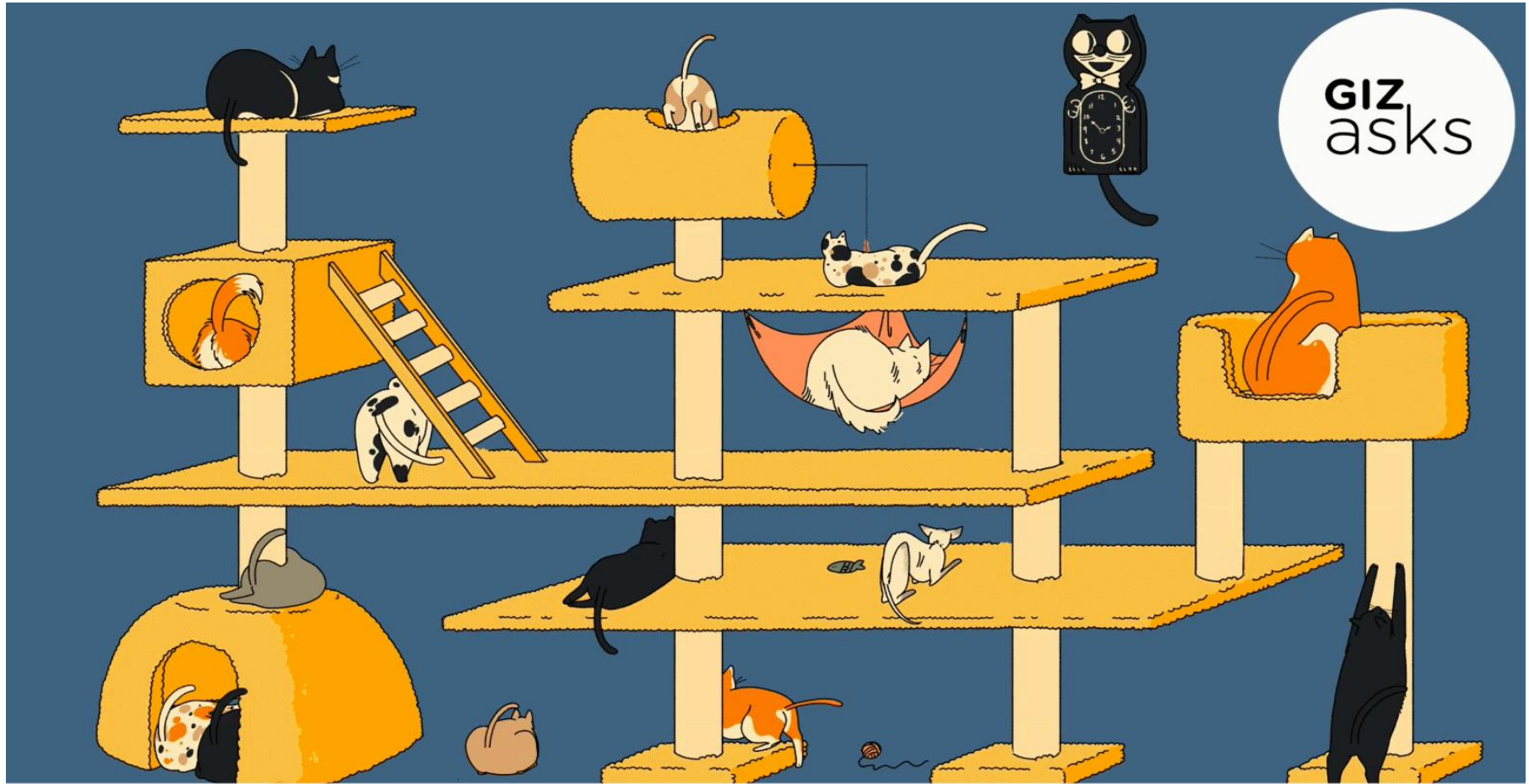
A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.



人类注意力机制



注意力机制的研究和应用

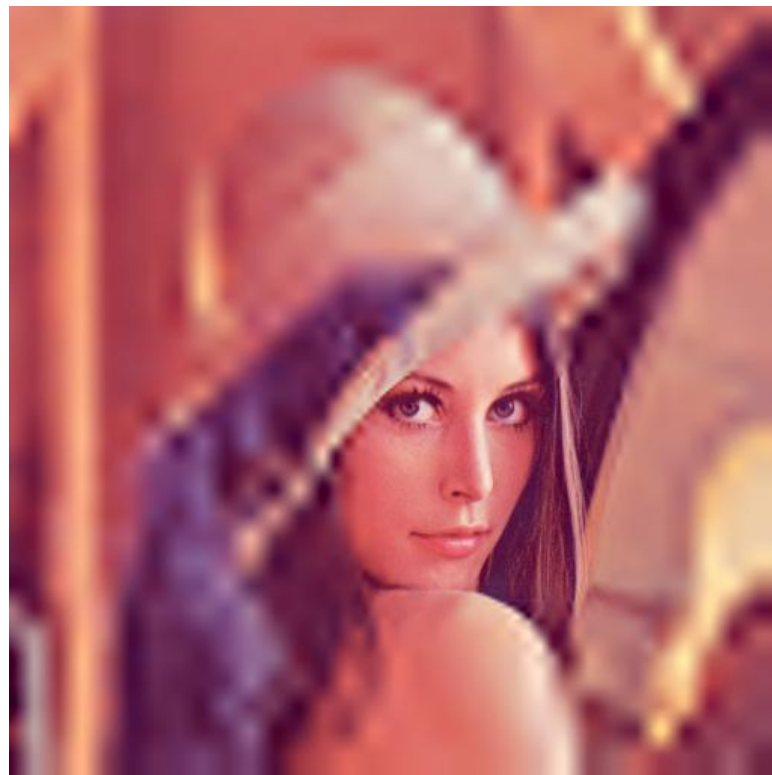
□ <http://saliency.mit.edu/datasets.html>

□ 数据库构建

- 眼动仪

□ 传统图像和视频处理中的应用

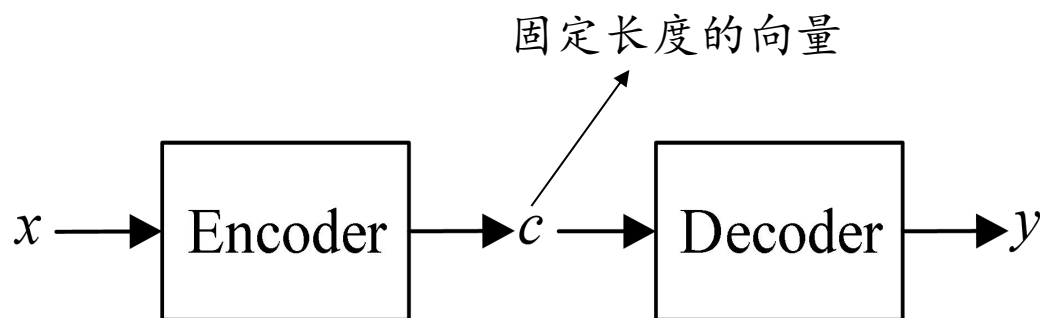
- 视频/图像质量评价
- 视频/图像的视觉编码优化
-



注意力机制

□ 注意力机制在在神经机器翻译领域的应用

- Dzmitry Bahdanau等人在神经机器翻译领域首先使用注意力机制
- 神经机器翻译主要以Encoder-Decoder模型为基础结构，首先利用Encoder对源语言输入序列 x 进行编码，形成一个语义向量 c ，然后利用Decoder对 c 进行解码，最终得到目标语言输出序列 y 。



源语言输入序列: $x = (x_1, x_2, \dots, x_n)$

目标语言输出序列: $y = (y_1, y_2, \dots, y_m)$

Dzmitry Bahdanau, et al., Neural machine translation by jointly learning to align and translate, 2015

注意力机制

□ 注意力机制在神经机器翻译领域的应用

- 在神经机器翻译中，Encoder一般采用RNN或者LSTM实现
 - 从统计角度，翻译相当于寻找译句 $\mathbf{y}$ ，使得给定原句 $\mathbf{x}$ 时条件概率最大

$$\operatorname{argmax}_{\mathbf{y}} p(\mathbf{y}|\mathbf{x})$$

- 得到上下文向量 $\mathbf{c}$ 的方法有很多，可以直接将最后一个隐状态作为上下文变量，也可对最后的隐状态进行一个非线性变换 $\sigma(\cdot)$ ，或对所有的隐状态进行非线性变换 $\sigma(\cdot)$

$$\mathbf{c} = h_T$$

$$h_t = f(x_t, h_{t-1})$$

$$\mathbf{c} = \sigma(h_T)$$

$$\mathbf{c} = \sigma(h_1, h_2, \dots, h_T)$$

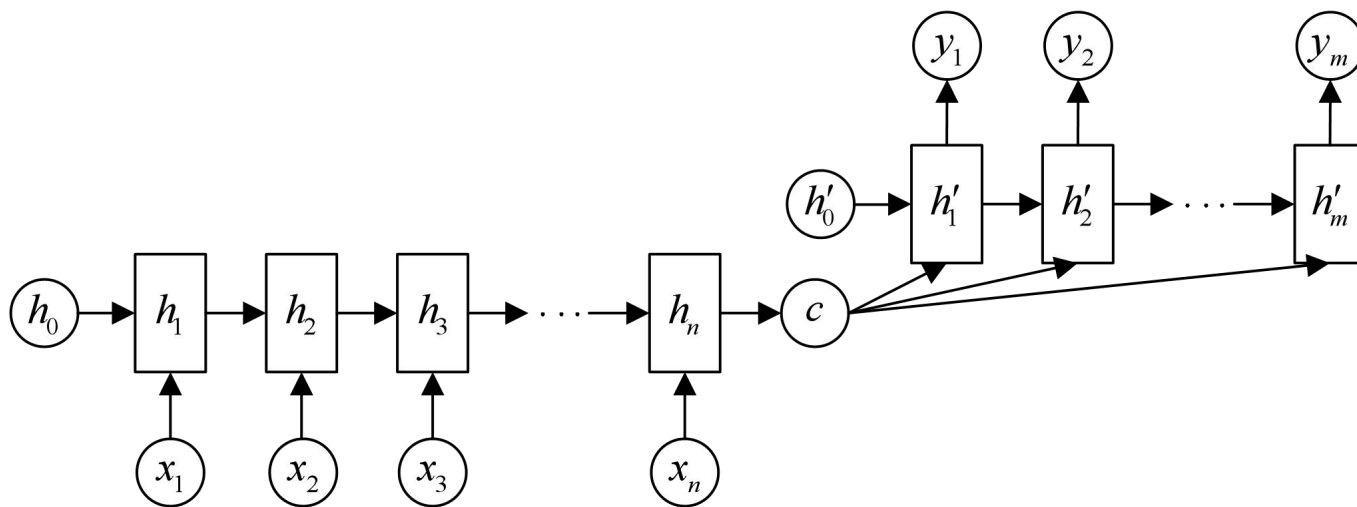
Dzmitry Bahdanau, et al., Neural machine translation by jointly learning to align and translate, 2015

注意力机制

□ 解码器

- 用给定的上下文向量 $\mathbf{c}$ 和之前已经预测的词 $\{y_1, \dots, y_{t-1}\}$

$$y_1 = G(\mathbf{c}) \quad y_2 = G(\mathbf{c}, y_1) \quad y_3 = G(\mathbf{c}, y_1, y_2)$$



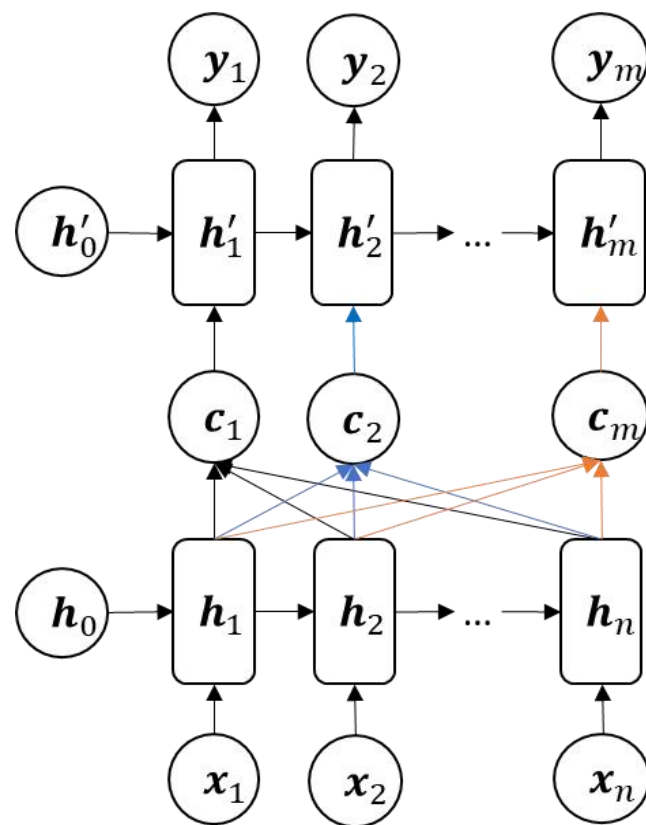
注意力机制

□ 现存问题

- 输入序列不论长短都会被编码成一个固定长度的向量表示，而解码则受限于该固定长度的向量表示
- 如果源语言输入序列比较长，这种结构会导致固定长度的语义向量 c 可能无法存储全部的语义信息，而注意力机制的提出则解决了这个问题

注意力机制

- ❑ 引入注意力机制的Encoder-Decoder结构
- ❑ 在解码过程中，每一步使用的是不同的语义向量 $c_j(j=1,2,\dots,m)$ ， c_j 会表达与当前输出 y_j 更相关的语义信息



注意力机制

□ 神经网络模型注意力机制

- 在这个新结构中，定义每个输出的条件概率为：

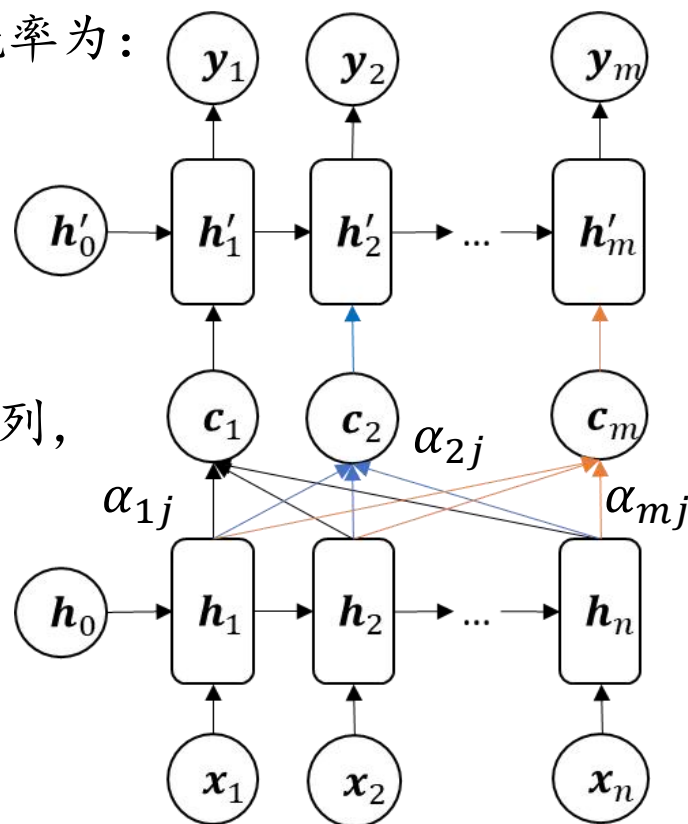
$$p(y_i | y_1, \dots, y_{i-1}, x) = g(y_{i-1}, h'_i, c_i)$$

- 其中 h'_i 为解码器RNN中的隐层状态：

$$h'_i = f(h'_{i-1}, y_{i-1}, c_i)$$

- 这里的上下文向量 c_i 取决于解码器状态序列，通过使用注意力系数 α_{ij} 对 h_j 加权求得

$$c_i = \sum_{j=1}^T \alpha_{ij} h_j$$



注意力机制

□ 注意力系数计算

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$$

$$e_{ij} = a(\mathbf{h}'_{i-1}, \mathbf{h}_j)$$



alignment mode

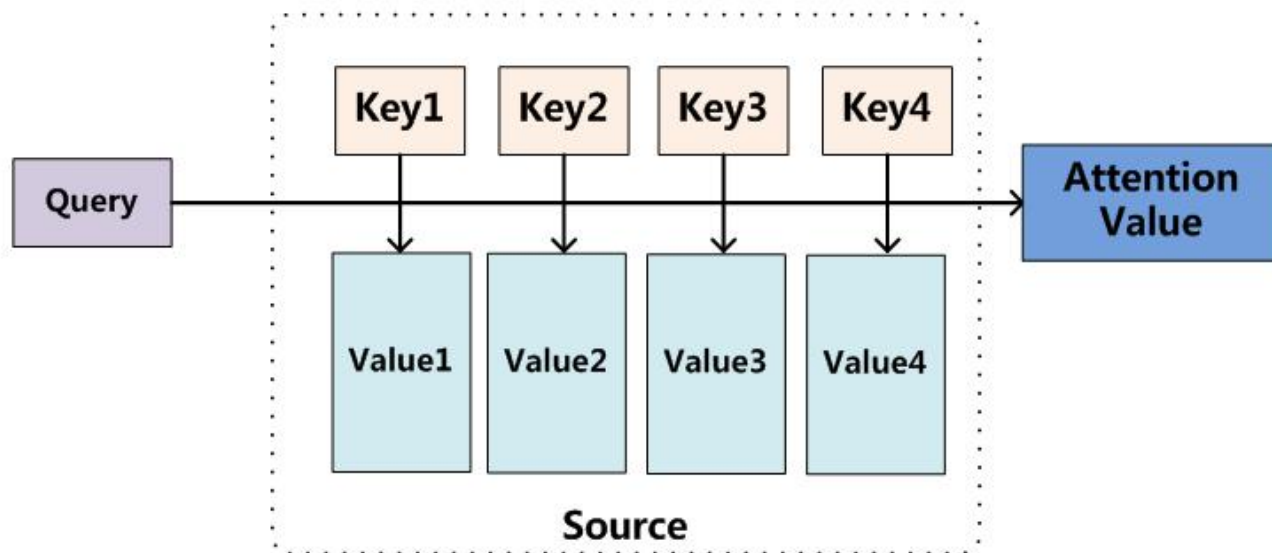
反映*i*位置的输入和*j*位置输出的匹配程度

□ 计算注意力系数的相似函数(alignment model)有以下几种:

$$a(\mathbf{h}'_{i-1}, \mathbf{h}_j) = \begin{cases} \mathbf{h}'_{i-1}{}^T \cdot \mathbf{h}_j \\ \frac{\mathbf{h}'_{i-1}{}^T \cdot W_\alpha \cdot \mathbf{h}_j}{W_\alpha \cdot [\mathbf{h}'_{i-1}{}^T, \mathbf{h}_j^T]^T} \\ v_\alpha \tanh(U_\alpha \mathbf{h}'_{i-1}{}^T + W_\alpha \mathbf{h}_j^T) \end{cases}$$

注意力机制的抽象理解

- Attention函数的本质可以被描述为一个查询（query）到一系列（键key-值value）对的映射



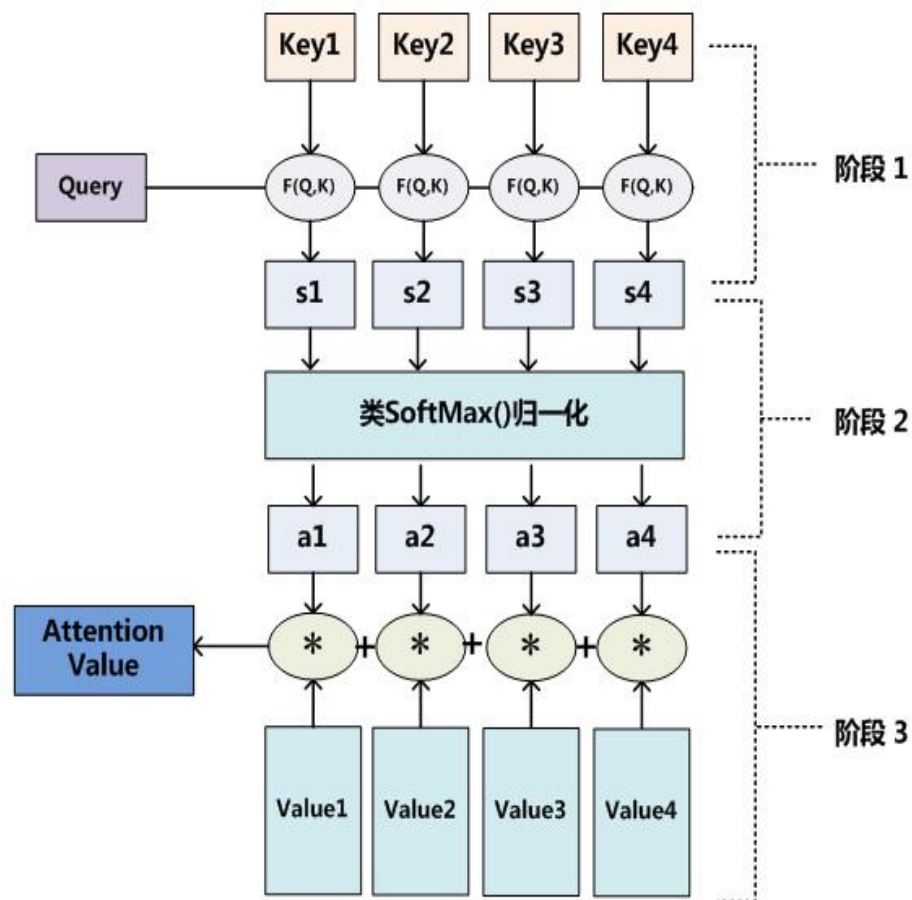
$$\text{Attention}(\text{Query}, \text{Source}) = \sum_{i=1}^{L_x} \text{similarity}(\text{Query}, \text{Key}_i) * \text{Value}_i$$

注意力机制的抽象理解

□ 注意力系数计算

- 阶段1：根据Query和Key计算两者的相似性或者相关性
- 阶段2：对第一阶段的原始分值进行归一化处理
- 阶段3：根据权重系数对Value进行加权求和，得到Attention Value

- Source(K, V)
- Target(Q)



注意力机制的抽象理解

□ 注意力系数计算

$$F(Q, K_i) = \begin{cases} Q^T \cdot K_i \\ Q^T \cdot W_\alpha \cdot K_i \\ W_\alpha \cdot [Q; K_i]^T \\ v_\alpha \tanh(U_\alpha K_i + W_\alpha Q) \end{cases}$$

点积模型

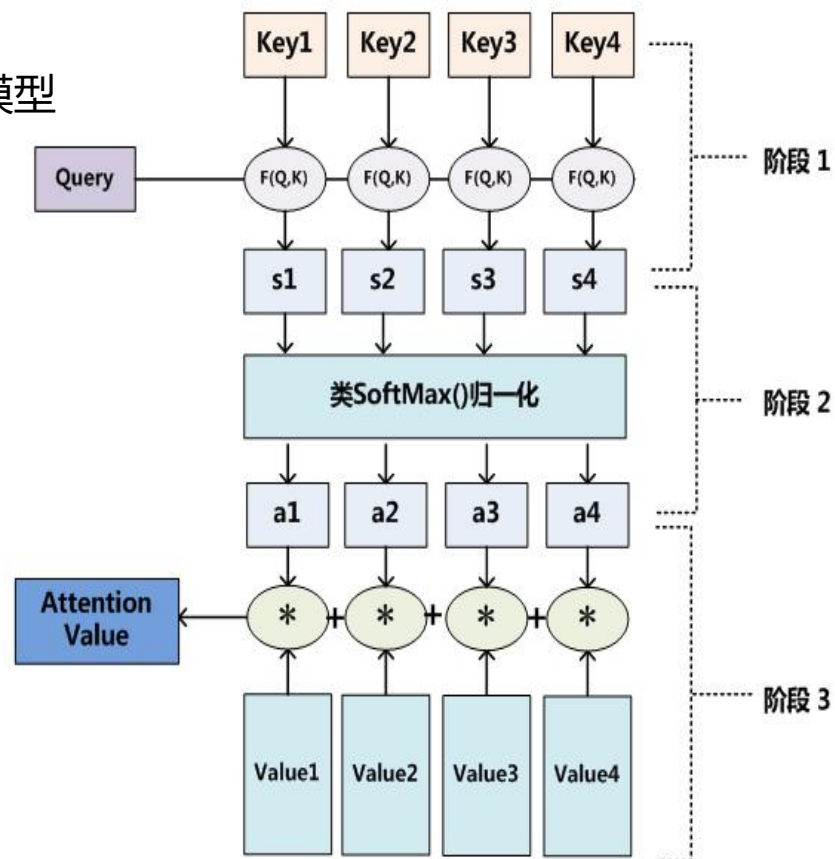
缩放点积模型

连接模型

加性模型

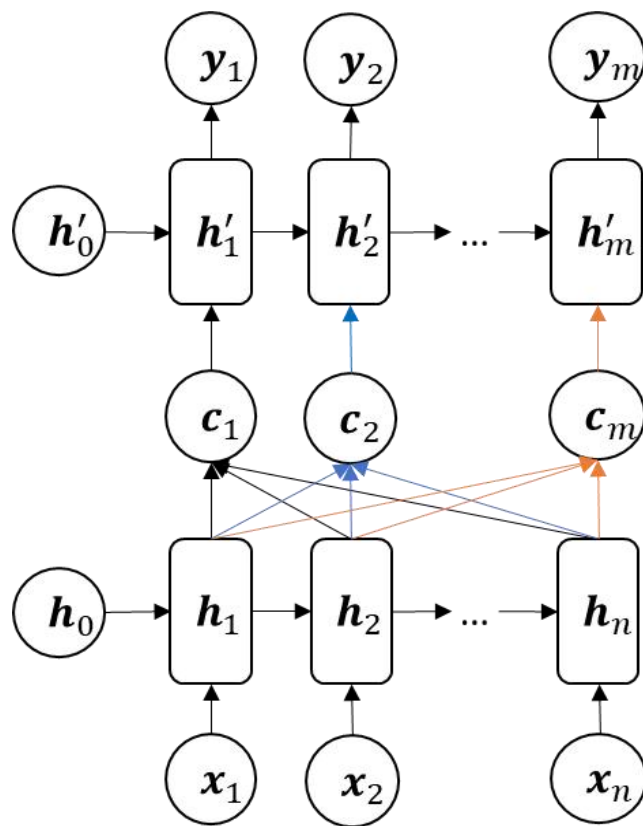
$$a_i = \frac{\exp(f(Q, K_i))}{\sum_j \exp(f(Q, K_j))}$$

$$\text{Attention}(Q, K, V) = \sum_j a_j V_j = c$$



注意力机制的分类

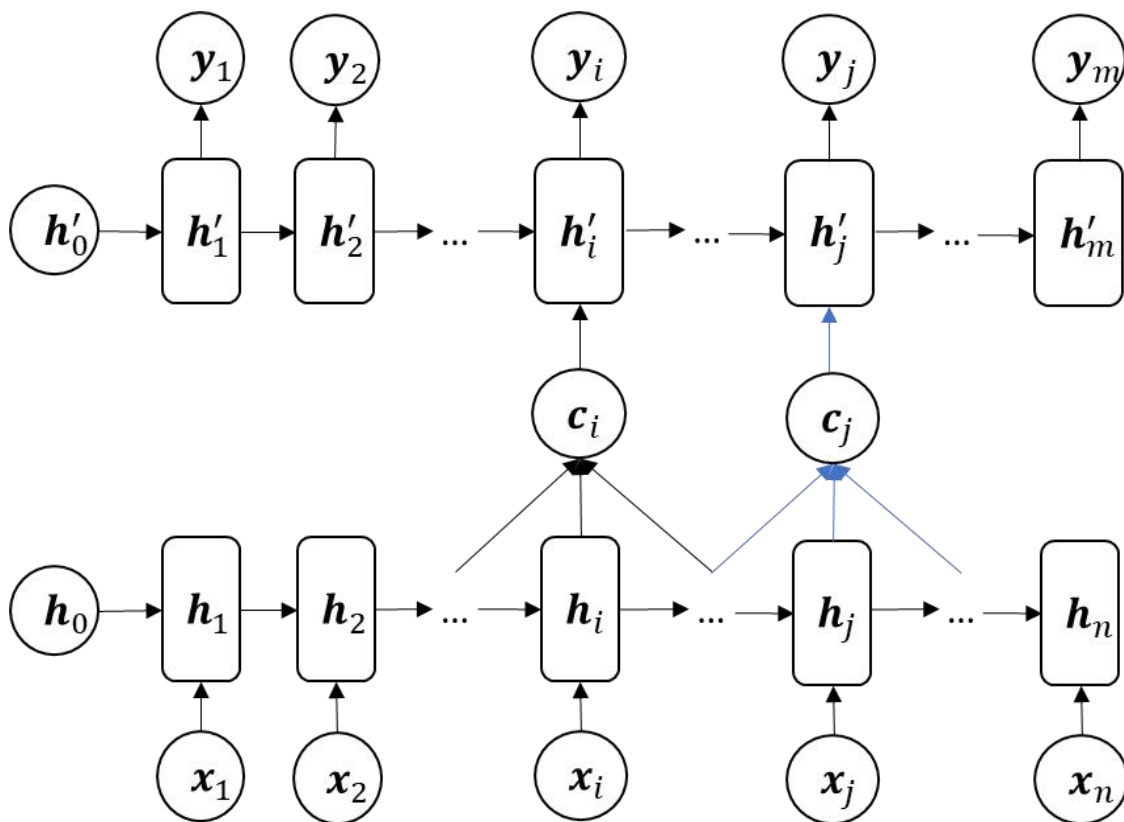
- 全局注意力：指Decoder端的注意力计算时要考虑Encoder端输入序列中所有的序列



注意力机制的分类

□ 局部注意力：是指Decoder端的注意力计算时仅考虑Encoder端输入序列中的部分序列。

- 首先预估一个对齐位置 p_i ，然后在 p_i 左右大小为 D 的窗口范围内来取序列进行注意力计算。



注意力机制的分类

□ 局部注意力的计算

$$c_i = \sum_{j=p_i-D}^{p_i+D} \alpha_{i,j} \mathbf{h}_j$$

$$\alpha_{i,j} = \frac{\exp(e_{ij}))}{\sum_{k=p_i-D}^{p_i+D} \exp(e_{ik}))} \exp\left(-\frac{(j-p_i)^2}{2(D/2)^2}\right)$$

$$e_{ij} = \mathbf{h}'_i{}^T \cdot \mathbf{W}_\alpha \cdot \mathbf{h}_j$$

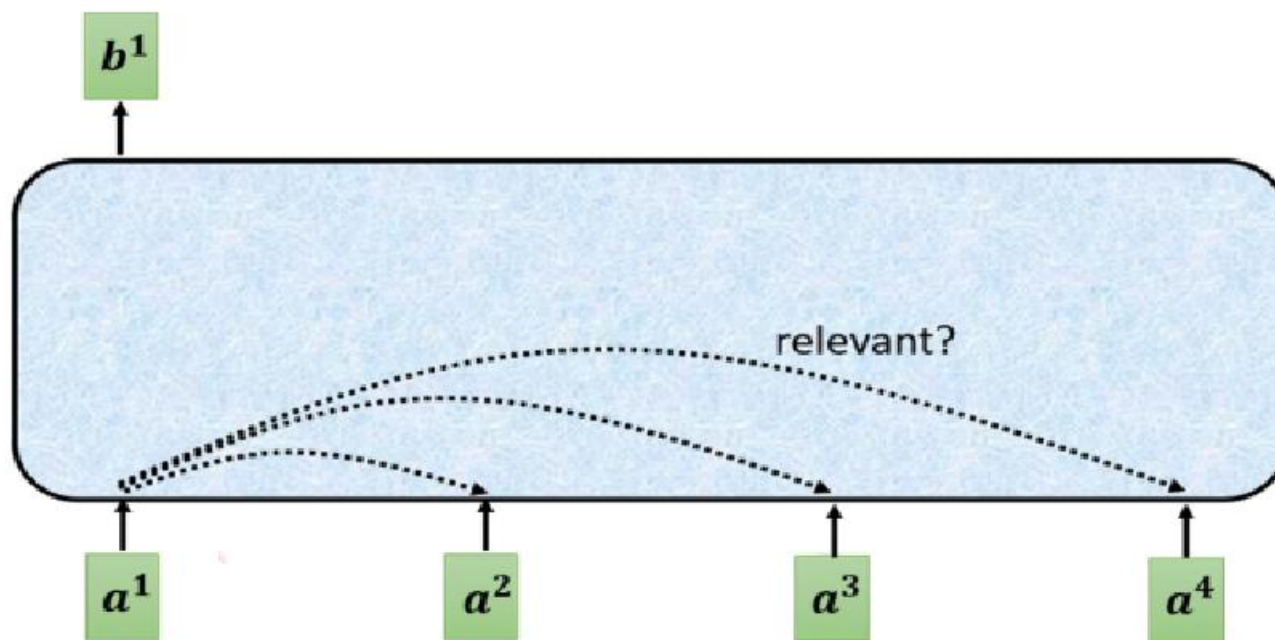
$$p_i = \mathbf{T}_x \cdot \text{sigmoid}(\mathbf{V}_p^T \tanh(\mathbf{W}_p \mathbf{h}'_j))$$

其中， p_i 是对齐位置， D 是窗口范围， $\alpha_{i,j}$ 是注意力系数， $\mathbf{T}_x$ 是Encoder端的向量表示， $\mathbf{W}_\alpha$ 、 $\mathbf{W}_p$ 和 $\mathbf{V}_p$ 是可学习的参数。

自注意力

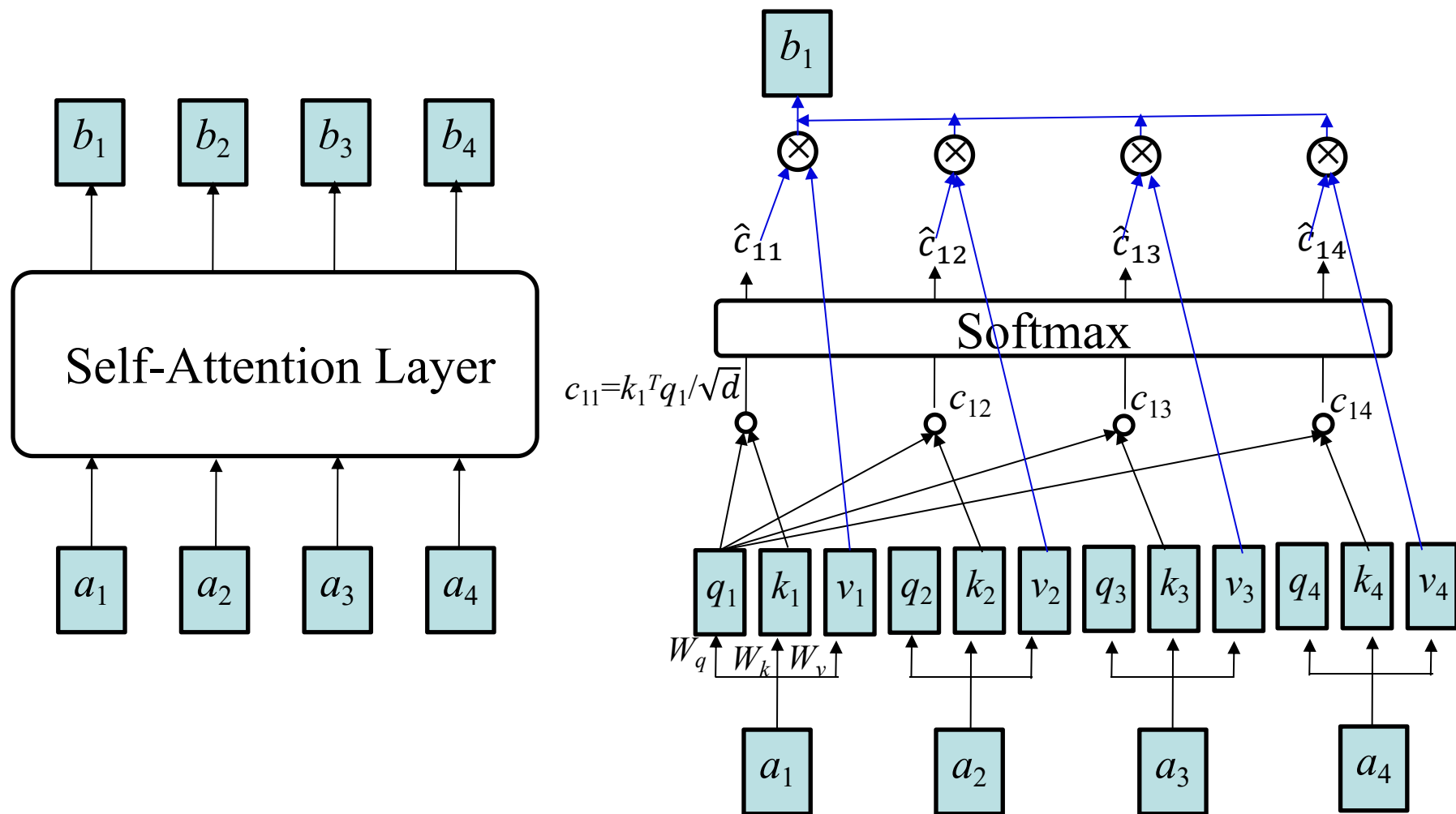
□ Self-attention layer in “Attention is all you need”

Self-attention



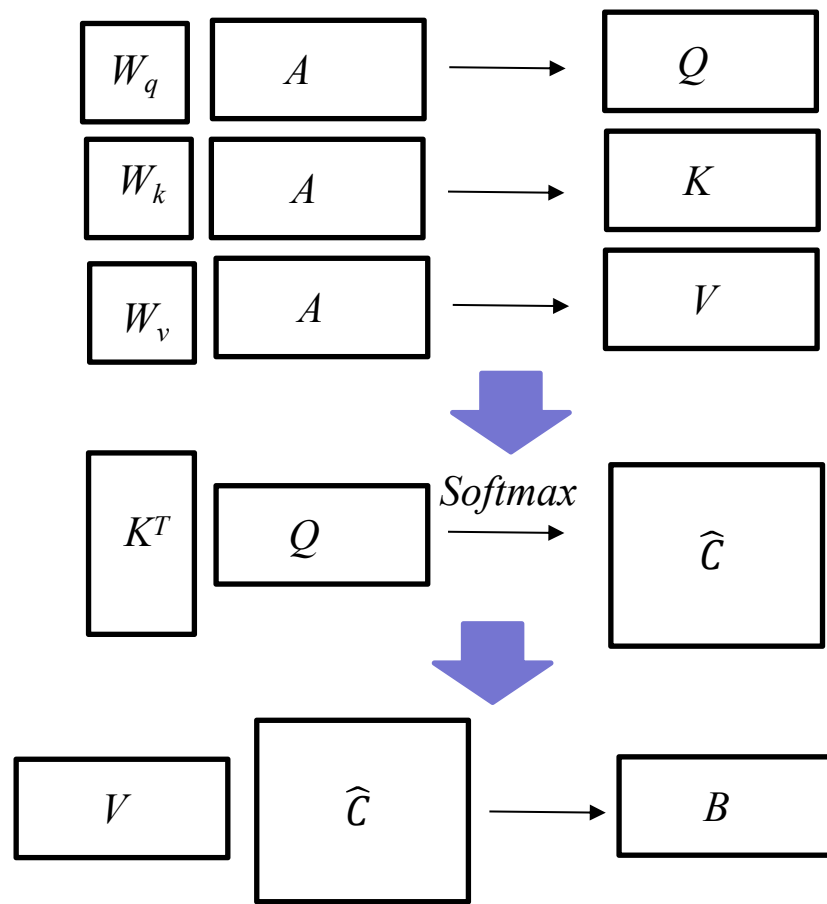
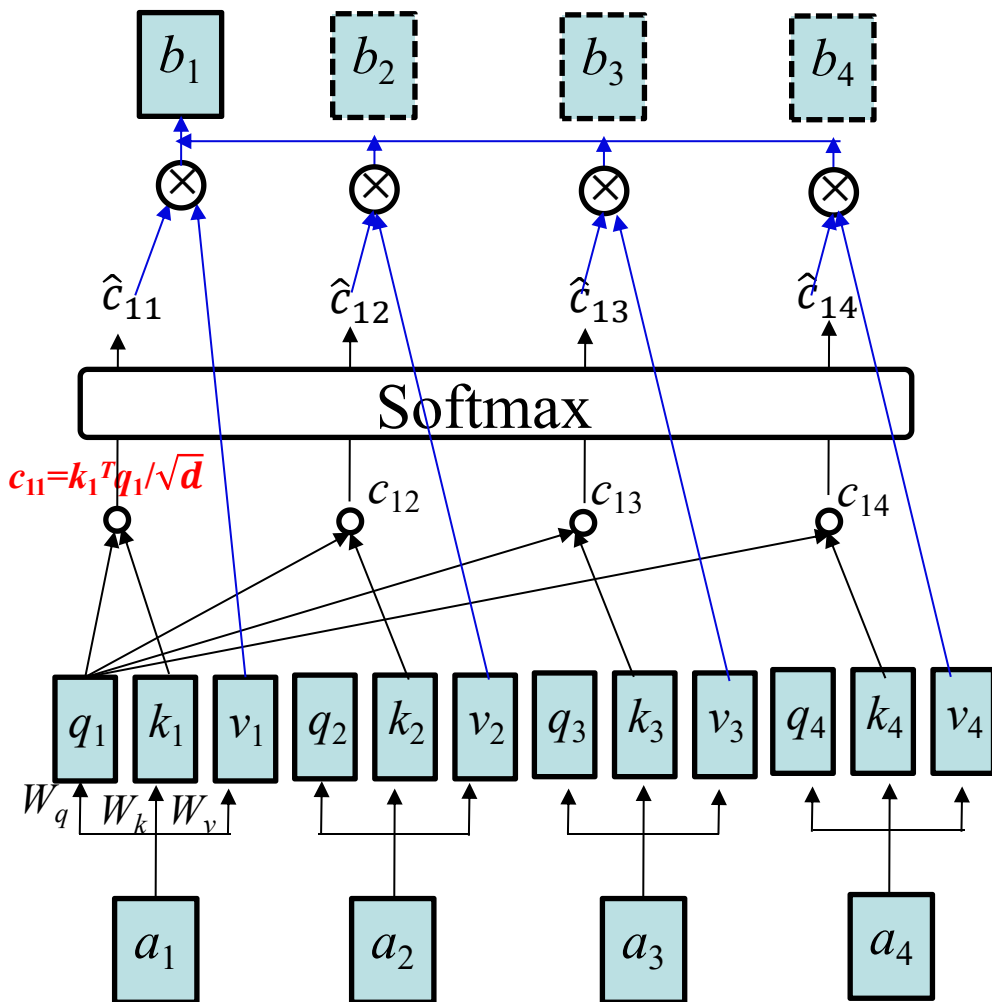
自注意力

□ Self-attention layer in “Attention is all you need”



注意力机制的抽象理解

❑ Self-attention layer in “Attention is all you need”



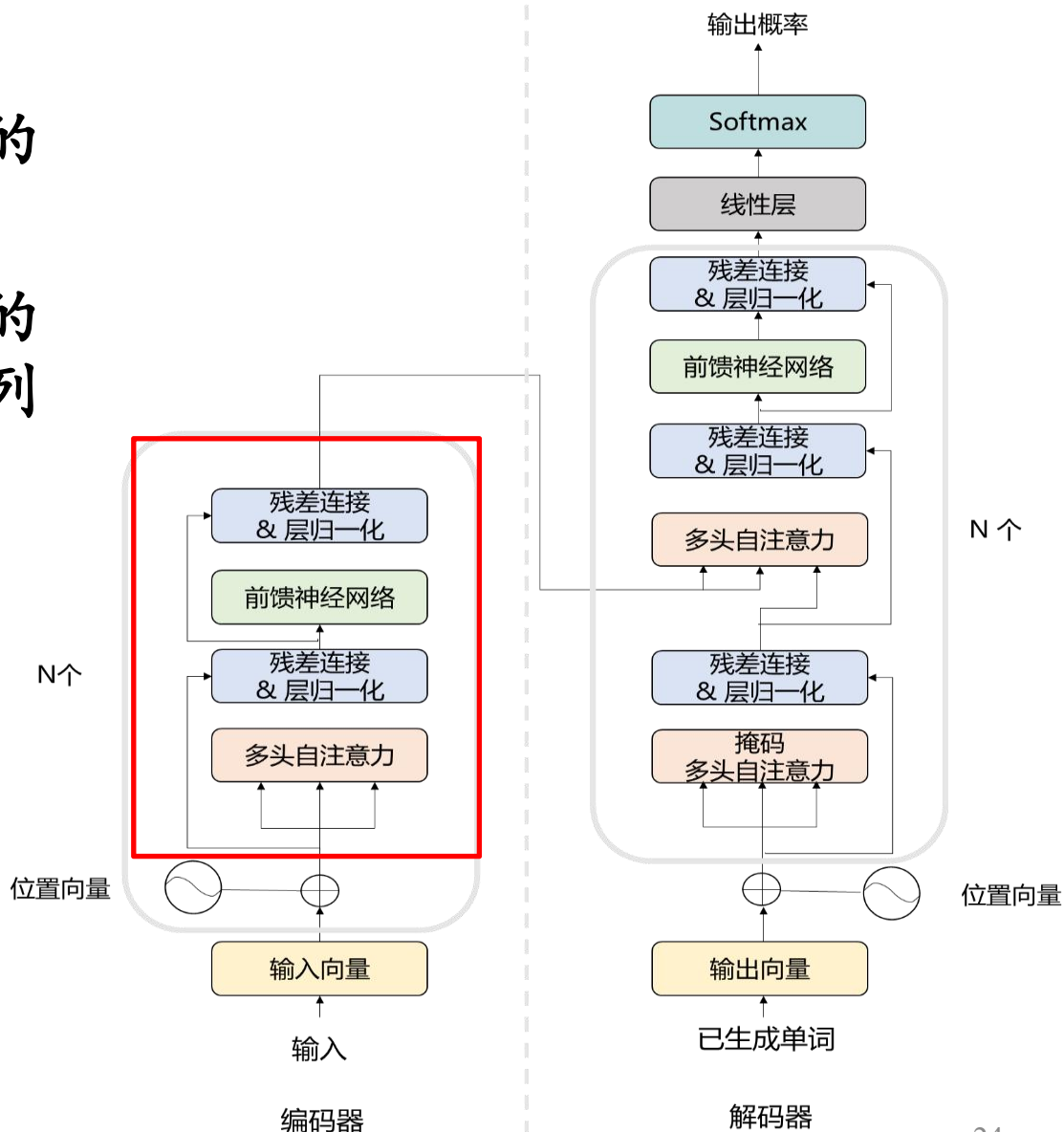


2

Transformer

Transformer的结构

- ❑ 编码器负责理解输入，为每个输入构造对应的语义表示
- ❑ 解码器负责以自回归的方式逐个生成输出序列中的元素



Transformer的结构

- ❑ Transformer的编码器由N个相同的层堆叠而成，每个层包含两个子层，分别是多头自注意力层和前馈神经网络层。其中，多头自注意力是Transformer的核心。
- ❑ 此外，在两个子层中，Transformer使用残差连接(Residual Connection)和层归一化(Layer Normalization, LN)机制进行性能优化。

Transformer的结构

□ 编码器中每一层的输出可以表示为：

$$output = LayerNorm(X + SubLayer(X))$$

其中， X 表示输入， $SubLayer$ 表示多头自注意力子层或者前馈神经网络子层， $LayerNorm(\cdot)$ 表示层归一化

Transformer的结构

- ❑ Transformer的解码器也是由N个相同的层堆叠而成。与编码器中每层的两个子层不同的是，**解码器的每层还包含第三个子层，对解码器的输出进行多头自注意力计算。**
- ❑ 另外，与编码器中的多头自注意力子层（第一个子层）不同的是，解码器中的第一个子层中的**多头自注意力使用了掩码（Mask）操作**，因此在解码过程中，**解码器只可以看到已经生成的解码序列，对未来即将生成的单词，需要进行掩码。**
- ❑ 在解码器的三个子层中，同样使用残差连接和层归一化机制进行了性能优化。

Transformer的输入编码

- ❑ Transformer首先将输入序列转换成词嵌入(Word Embedding)向量(简称词向量)。在实现过程中，词向量可以随机初始化后随网络训练得到，也可以加载预先训练好的词向量表示，如Word2Vec、Glove等。
- ❑ 为了使得Transformer架构能够在编码输入序列时包含位置信息，在词向量之外，还增加了位置嵌入(Position Embedding)向量(简称位置向量)

Transformer的输入编码

- 采用不同频率的正弦和余弦函数得到输入序列中每个标记的位置向量：

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

- 其中， pos 表示单词在输入序列中的位置， d_{model} 表示模型输出的维度，输入的词向量和位置向量的维度与之相等，一般取512，维度 $2i$ 表示偶数维度， $2i+1$ 表示奇数维度。

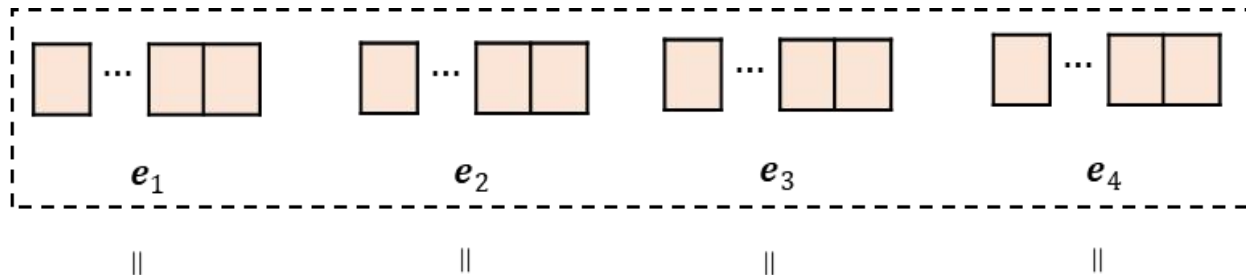
pos/i	0	1	2	3	4		510	511
0	sin()	cos()	sin()	cos()	sin()		sin()	cos()
1	sin()	cos()	sin()	cos()	sin()		sin()	cos()
2	sin()	cos()	sin()	cos()	sin()		sin()	cos()
.....								

Transformer的输入编码

- Transformer的输入编码最终由输入序列中标记的词向量和位置向量求和得到。举例说明，假设输入序列为 $x=(x_1, x_2, x_3, x_4)$ ，序列对应的词向量为 $w=(w_1, w_2, w_3, w_4)$ ，位置向量为 $p=(p_1, p_2, p_3, p_4)$ ，那么输入序列在Transformer中的输入编码为： $e=(e_1, e_2, e_3, e_4)$ ，其中 $e_i=w_i+p_i$ 。

Transformer的输入编码

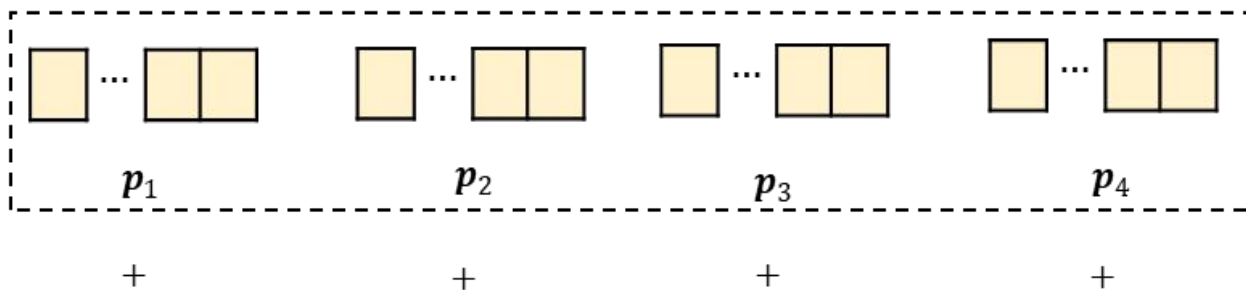
输入编码



不同频率正
弦/余弦函数



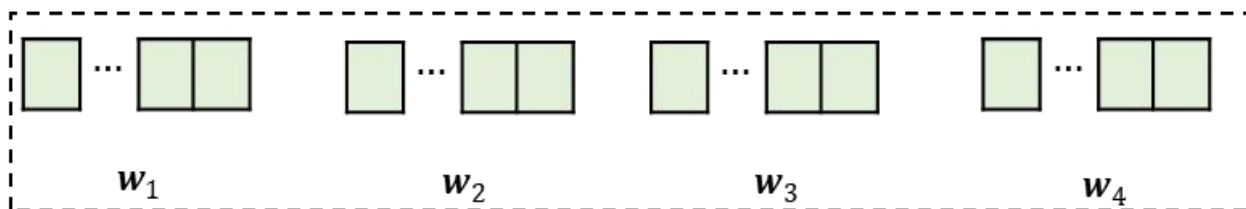
位置向量



预训练或随
机初始化后
随网络训练



词向量



输入序列



Transformer中的自注意力机制

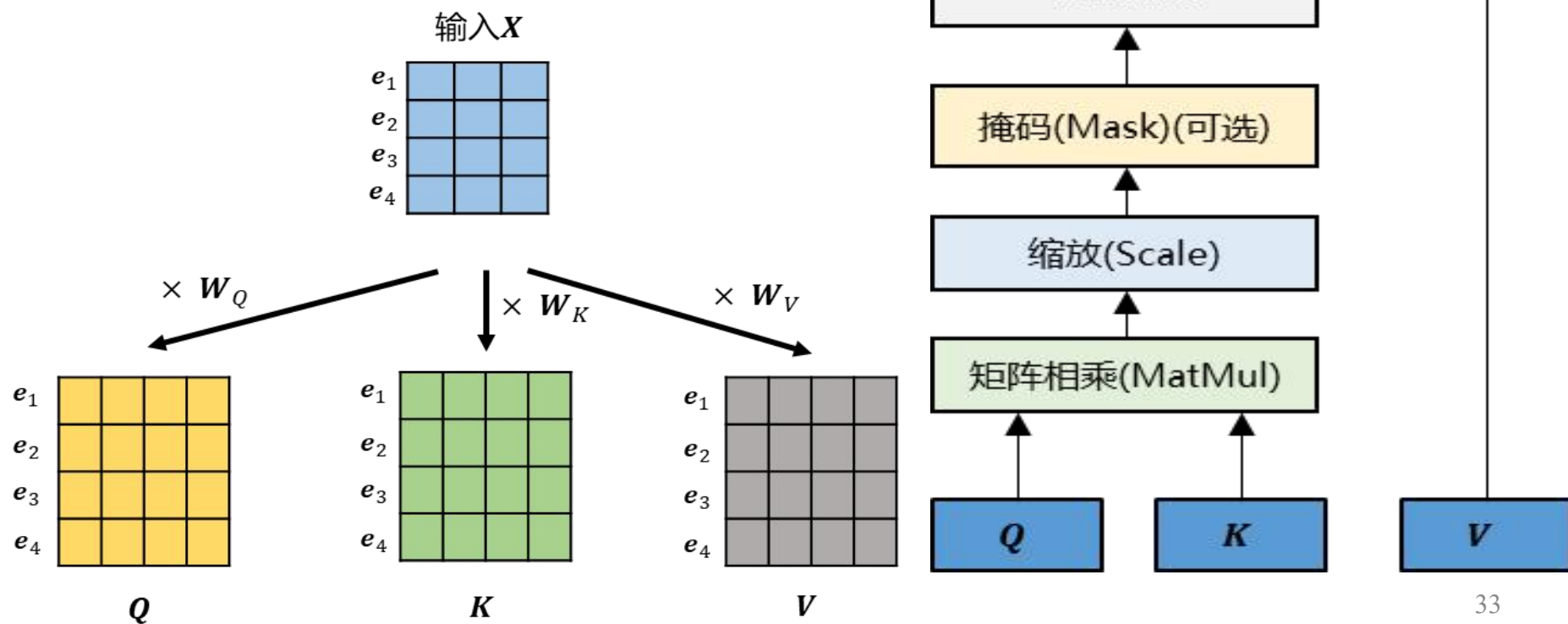
- 在Transformer中，提出了一种可缩放的点积注意力（Scaled Dot-product Attention），也就是自注意力。
- 该注意力机制的输入由查询向量 Q 、维度为 d_k 的键向量 K 和维度为 d_v 的值向量 V 构成。接下来，计算查询向量 Q 和键向量 K 的点积（矩阵相乘），并除以 $\sqrt{d_k}$ 进行缩放，之后在编码器中直接跳过掩码操作，而在解码器中需要进行掩码操作，最后使用**Softmax**函数来获得值向量 V 的权重系数，再与值向量 V 进行矩阵相乘得到最终结果。

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

Transformer中的自注意力机制

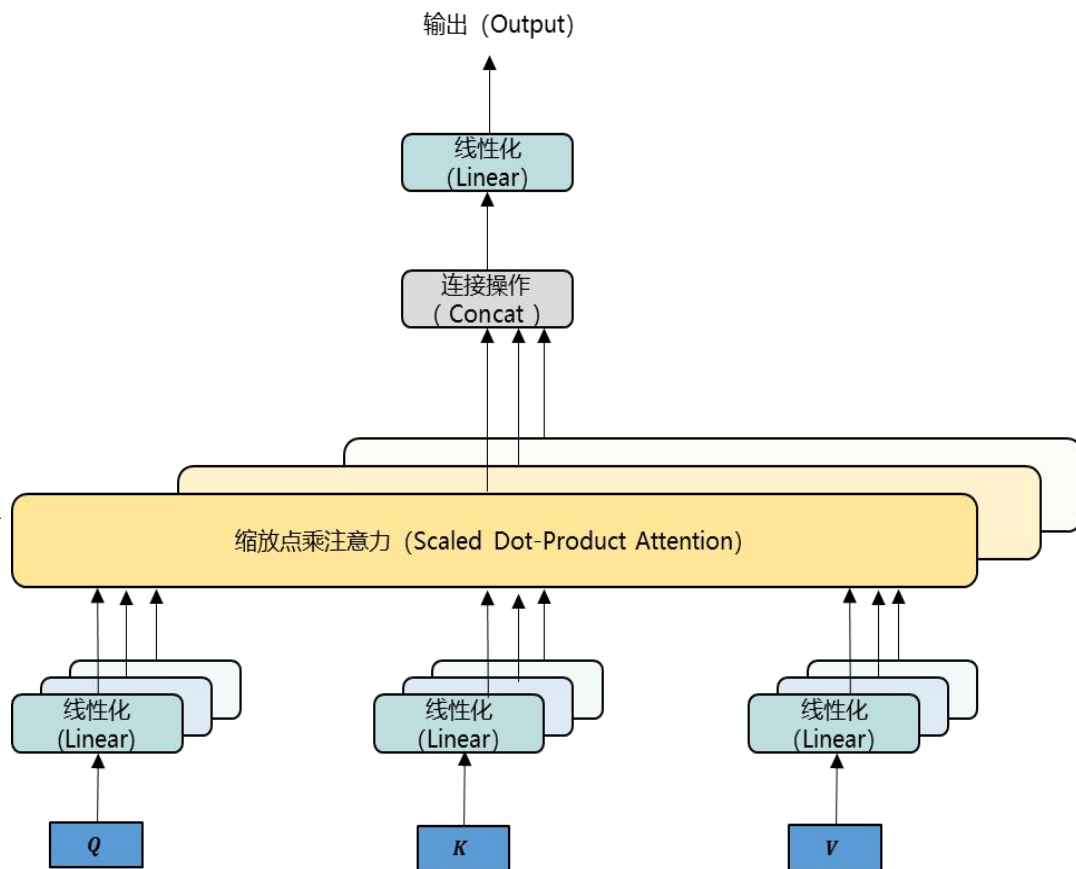
- 具体地，在实践中，首先将输入X分别通过线性变换矩阵 W_Q 、 W_K 和 W_V 映射成查询矩阵Q、键矩阵K和值矩阵V。之后根据公式计算注意力输出：

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$



Transformer中的自注意力机制

- Transformer可以并行地执行自注意力机制，产生多个输出值，再将这些值拼接起来进行再次投影。因此，在自注意力机制的基础上设计了多头自注意力机制（Multi-head Self-attention Mechanism）



Transformer中的自注意力机制

□ 多头自注意力机制 (h 头) 的计算公式如下:

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O$$

$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$, W_i^Q 、 W_i^K 、 W_i^V 和 W^O 是可学习的参数, $W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{model} \times d_v}$, $W^O \in \mathbb{R}^{hd_v \times d_{model}}$ 。通常取 $h = 8$, 那么 $d_k = d_v =$

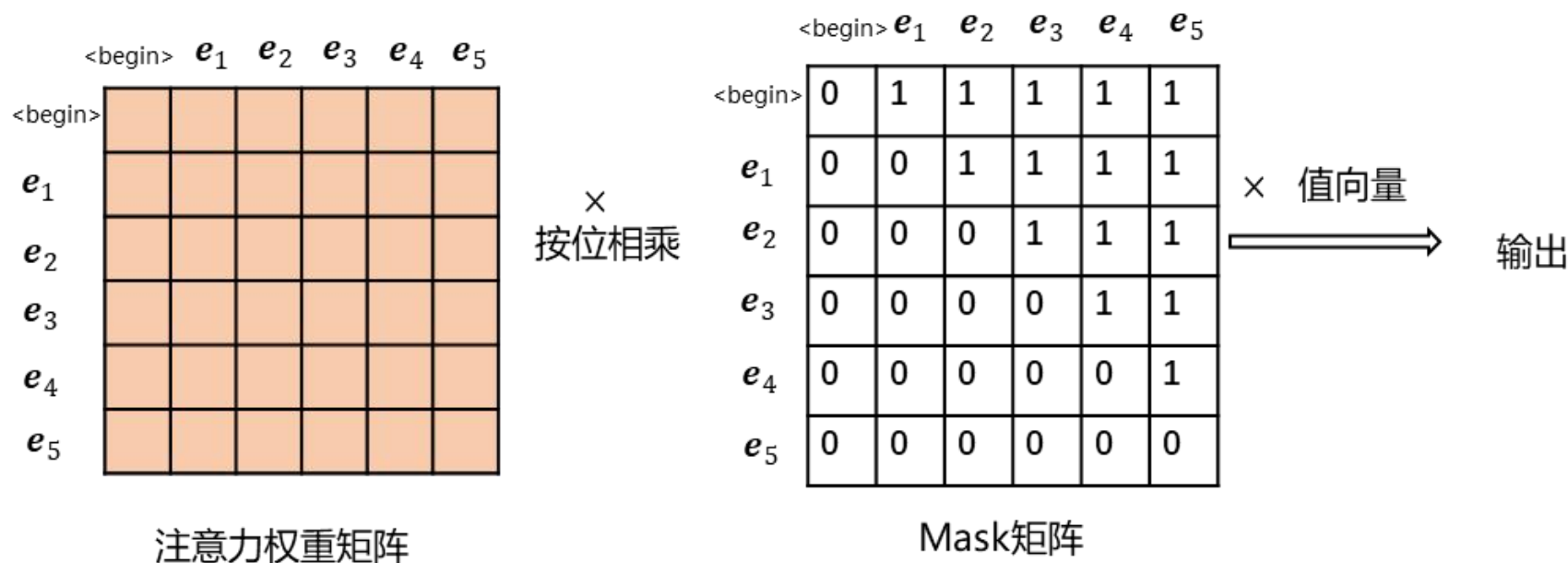
$$\frac{d_{model}}{h} = \frac{512}{8} = 64。$$

Transformer中的自注意力机制

- 前面提到，在解码器中第三个子层是带有掩码操作的多头自注意力。就是说在 t 时刻解码到当前单词的时候，通常需要已经产生的单词作为输入历史信息对当前单词进行概率预测。
- 为了避免模型提前看到 t 时刻之后的单词，需要对输入序列 t 时刻之后的单词进行掩码，也就是带有掩码操作的多头自注意力机制。

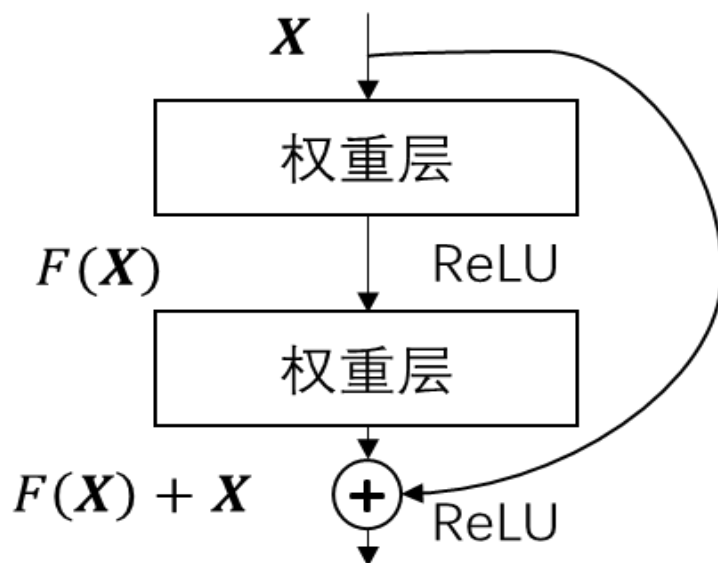
Transformer中的自注意力机制

- 具体来说，对于前文中的输入X，假设通过输入编码得到 $e=\{e_1, e_2, e_3, e_4, e_5\}$ ，那么在带有掩码的多头注意力机制中，会有一个掩码矩阵，当解码到第*i*个单词的时候，第*i*+1个单词被掩盖，继而进行后面的操作得到输出。



Transformer中的残差连接

- 残差连接通过从输入直接引入一个短连接到非线性层的输出，实现输入跨层。残差连接会将输入数据和输出数据相加，然后使用一个激活函数处理。这样以来，即使网络层数很深，信息也可以很容易地传递到后面的网络层中，从而有效解决了梯度消失问题。



Transformer中的层归一化

- ❑ 层归一化的计算是针对每个样本的所有特征进行的，而不是针对每个特征或者每个批次进行的。
- ❑ 层归一化会计算某个网络层中所有特征的均值和方差，并将这些统计量用于标准化该层的输出。层归一化是一种简单有效的技术，用来帮助深度神经网络更好地学习特征表示，提高模型的性能和泛化能力。

Transformer中的层归一化

□ 层归一化的计算公式：

$$\text{layernorm}(\mathbf{u}) = \frac{\mathbf{u} - E(\mathbf{u})}{\sqrt{\text{Var}(\mathbf{u}) + \varepsilon}} \gamma + \beta$$

□ 其中， $\mathbf{u}$ 表示层归一化的输入向量， $E(\mathbf{u})$ 是向量的均值， $\text{Var}(\mathbf{u})$ 是向量的方差， ε 是用来保证分母不为0的小数， γ 和 β 是可学习参数。

Transformer中的前馈神经网络

- 前馈神经网络，它由两个线性变换组成，中间由ReLU激活函数进行激活，对应的计算公式如下：

$$FFN(X) = \max(0, XW_1 + b_1)W_2 + b_2$$

- 其中， W_1 、 W_2 是前馈神经网络的权重， b_1 、 b_2 是前馈神经网络的偏置。

基于Transformer的大规模预训练模型

- ❑ Transformer能够并行处理较长的输入序列，这一优点吸引了大量研究人员的关注，基于Transformer的大规模预训练模型应运而生。
- ❑ 基于Transformer的大规模预训练模型通常是在大规模文本语料库上进行无监督学习而成，这种预训练方式使得模型能够学习到丰富的语义表示，继而应用到各项自然语言处理任务中，如OpenAI的GPT系列、Google的BERT系列，均已在机器翻译、文本摘要、情感分析、问答等多个自然语言处理任务中取得了优秀的实验结果。



3

GPT系列模型

GPT预训练模型

- ❑ 生成式预训练Transformer（Generative Pre-trained Transformer, GPT）是一种基于Transformer的自回归语言模型，即模型能够迭代地根据已经生成的单词来逐个预测后面的单词。
- ❑ GPT由OpenAI公司在2018年提出，广泛应用于自然语言处理中的各项任务，如文本分类、文本蕴含、文本相似度计算和问答等。
- ❑ GPT模型已经推出了多个版本，包括GPT-1、GPT-2和GPT-3等，它们不断刷新在各项自然语言处理任务上的最好实验结果，成为自然语言处理领域的标志性模型。

GPT-1

- ❑ 自然语言理解包括多种不同的任务，如文本分类、文本蕴含、语义相似度计算和问答等。尽管大型未标记文本语料库很丰富，但是用于学习特定任务的标记文本语料库数量很少，这为训练特定任务对应的模型带来了挑战。
- ❑ GPT-1模型提出在大规模未标记语料库上训练生成式语言模型，并在下游任务上进行微调以提升下游任务的性能，减轻模型对标记数据的依赖。
- ❑ GPT-1采用无监督预训练和监督微调相结合的方式，目标是学习一个通用表示，并将其迁移到更广泛的应用上去。
- ❑ 在模型架构上，GPT-1基于Transformer构造，这是因为与其他卷积神经网络或者循环神经网络相比，Transformer提供了效率更高的方法来处理文本中的长期依赖关系。

GPT-1

□ 无监督预训练

- 给定一个无监督的语料数据集 $U=\{u_1, u_2, \dots, u_n\}$, GPT-1使用标准的语言模型进行训练, 也就是最大化如下似然估计:

$$L_1(U) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \theta)$$

- 其中, k 表示上下文窗口的大小, P 表示条件概率, θ 是网络的参数, 采用梯度下降法进行训练。

GPT-1

□ 无监督预训练

- GPT-1使用一个多层Transformer解码器进行语言建模，该模型对输入上下文使用多头自注意力机制，然后使用前馈神经网络预测目标单词的概率分布，建模过程如下：

$$\mathbf{h}_0 = \mathbf{U}\mathbf{W}_e + \mathbf{W}_p$$

$$\mathbf{h}_l = \text{transformer_block}(\mathbf{h}_{l-1})$$

$$P(\mathbf{U}) = \text{softmax}(\mathbf{h}_n \mathbf{W}_e^T)$$

- 其中， $\mathbf{W}_e$ 是一个单词向量矩阵， $\mathbf{W}_p$ 是位置向量矩阵， $\mathbf{U}$ 表示输入文本的上下文向量， n 是网络层数。

GPT-1

□ 有监督微调阶段

- 在使用上述方法进行预训练之后，GPT-1采用有监督微调方法将参数调整到更适合下游任务的状态。
- 假设现在有某个有标记的样本集C，其中每个样本由一系列的输入单词 $\{x^1, x^2, \dots, x^m\}$ 和一个标签 y 组成，那么输入单词序列首先被送入在前一个阶段预训练好的模型中得到一个状态向量 $\mathbf{h}_l^m$ ，接着被送入一个线性层进行结果预测：

$$P(y|x^1, x^2, \dots, x^m) = \text{softmax}(\mathbf{h}_l^m \mathbf{W}_y)$$

GPT-1

□ 有监督微调阶段

- 基于此，有监督微调阶段的目标函数就是：

$$L_2(\mathbf{C}) = \sum_{(x,y)} \log P(\mathbf{y} | \mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^m)$$

- 与此同时，GPT-1的设计人员发现，在有监督微调阶段将语言模型作为微调的辅助目标能够进一步提升模型的泛化能力，并且可以加速收敛帮助学习。最终，在有监督微调阶段，模型的训练目标变为：

$$L_3(\mathbf{C}) = L_2(\mathbf{C}) + \lambda * L_1(\mathbf{C})$$

其中 λ 是权重系数。

GPT-1

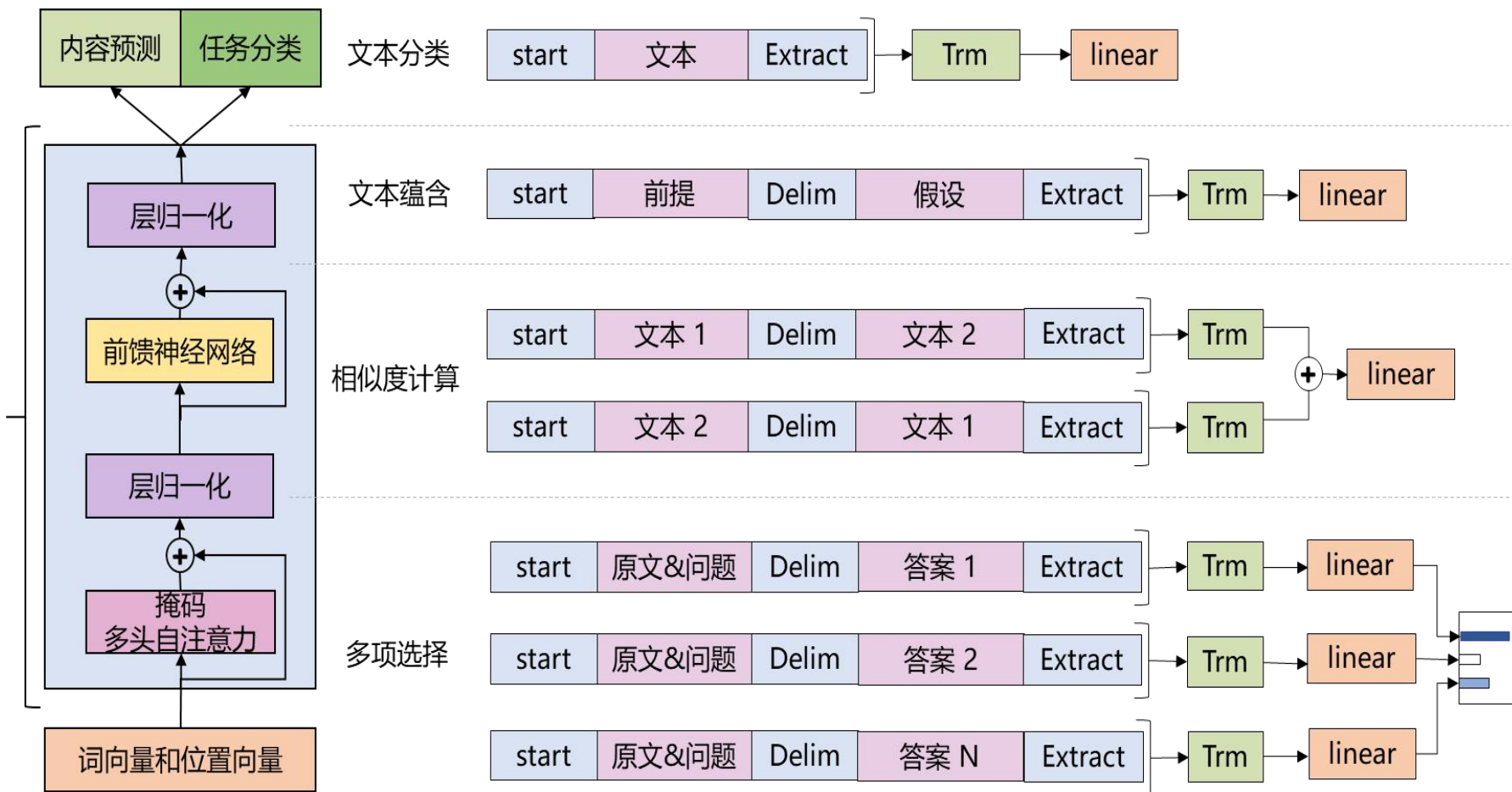
□ 任务适应

- 对于像文本分类一样的任务，可以按照上述有监督微调阶段所提到的方法进行微调，但是像问答、文本蕴含等自然语言理解任务，则需要在输入阶段进行相应的设计和修改，才可以将GPT模型进行应用。
- 在微调阶段，所有任务的输入都增加了特殊的Token作为输入的开始[start]和结束[extract]。

□ 任务适应

- 对于文本蕴含任务，设计者将前提和假设进行拼接，然后在二者之间增加了一个特殊标记“Delim”。
- 对于文本相似度计算任务，由于被计算相似度的两个句子之间没有前后顺序关系，所以对同一对数据进行了不同顺序的拼接，最后使用线性化得到相似度得分。特别地，输入时两个句子之间增加了特殊标记“Delim”，用来区分前后两个句子。
- 对于多项选择的问答任务，将每个候选答案与问题和原文片段进行拼接，得到相应的向量表示，最后使用Softmax函数在候选答案范围中进行结果概率预测。

GPT-1



GPT-1

- ❑ 在模型实现细节上，GPT-1很大程度上遵循了原始的Transformer工作模式，训练了一个具有掩码自注意力机制的12层仅包含解码器的Transformer。对于前馈神经网络，使用了3072维的内部状态，使用最大学习率为 2.5×10^{-4} 的Adam优化方案。
- ❑ GPT-1模型在无监督训练阶段采用BooksCorpus数据集，这个数据集包含各种风格的7000多本在当时尚未出版的图书数据，主题类型包括冒险、奇幻等。至关重要的是，它包含连续的长文本。另一个数据集是Word Benchmark数据集。

GPT-1

- GPT-1在12个自然语言理解任务中的9个任务上达到了当时最好的实验结果，比如，在常识推理和问答任务上实现了8.9%的绝对改进，在文本蕴含任务上实现了1.5%的绝对改进。

GPT-2

- ❑ GPT-2在GPT-1的基础上进行了更通用的迁移方法探索，证明了语言模型可以在零样本（Zero-shot）情况下执行下游任务，而无需任何参数或者架构的修改。
- ❑ GPT-2提供了一种更为灵活和通用的形式来指定任务、输入和输出，避免了算法级别的任务定制。如在机器翻译任务中，模型的输入可以被设定为“请翻译为法语、英语原文内容、对应法语内容”；在机器阅读理解任务中，训练样本可以被写成“问题的答案、问题、原文、答案”。
- ❑ GPT-2的思想本质上就是早期的指令微调（Prompt Finetuning）方法，这种做法可以使得模型学习在遇到类似的提示语之后，应该输出什么样的内容。

GPT-2

- ❑ 为了支持多个零样本任务，GPT-2需要在预训练阶段学习尽可能丰富的数据，因此OpenAI自建了高质量的WebText数据集，只保留了人工过滤过的网页，最终包含4500万个链接。
- ❑ 在模型实现上，GPT-2仍然使用Transformer作为主干模型，与GPT-1的整体架构类似，只进行了少量修改，包括层归一化被移到每个块的输入部分，起到类似预激活的作用，在最终的自注意力块之后又增加了额外的层归一化，同时Transformer的Decoder层数从GPT-1的12层增加到了24层、36层和48层。
- ❑ GPT-2在大多数的零样本任务上取得了当时最好的实验结果。

GPT-3

- ❑ GPT-3的主要目标是使用更少的领域数据，且不需要经过微调去解决问题。它沿用GPT-2的模型和训练方法，将模型参数大小从GPT-2的15亿个升级到1750亿。
- ❑ 在几十个自然语言处理数据集上对GPT-3进行了评估，包括三种设置：（1）零样本学习（Zero-shot Learning）：不允许展示具体的任务样本，只告知模型自然语言表示的指令；（2）单样本学习（One-shot Learning）：只允许向模型展示一个样本；（3）小样本学习（Few-shot Learning）：允许尽可能多的向模型展示样本（大概在10-100个之间）。

GPT-3

- ❑ 在开发GPT-3的过程中，研究人员发现，模型增大之后，引入一些质量较差的数据带来的负面影响变小了，因此与GPT-1和GPT-2相比，GPT-3开始使用Common Crawl数据集进行训练。
- ❑ 最终，它结合了清洗好的Common Crawl数据和已有的高质量数据，使用了超过45TB的文本数据作为训练数据，并进行了不同权重的采样使用。

GPT-3

零样本

给定任务相关的自然语言描述，
模型预测答案。梯度不进行更新。

1	Translate English to French	← 任务描述
2	Cheese=>	← 提示 (prompt)

单样本

除了任务描述之外，模型能够看到一个本任务的具体样例。梯度不进行更新。

1	Translate English to French	← 任务描述
2	Sea otter=>loutre de mer	← 样例 (example)
3	Cheese=>	← 提示 (prompt)

小样本

除了任务描述之外，模型能够看到多个本任务的具体样例。梯度不进行更新。

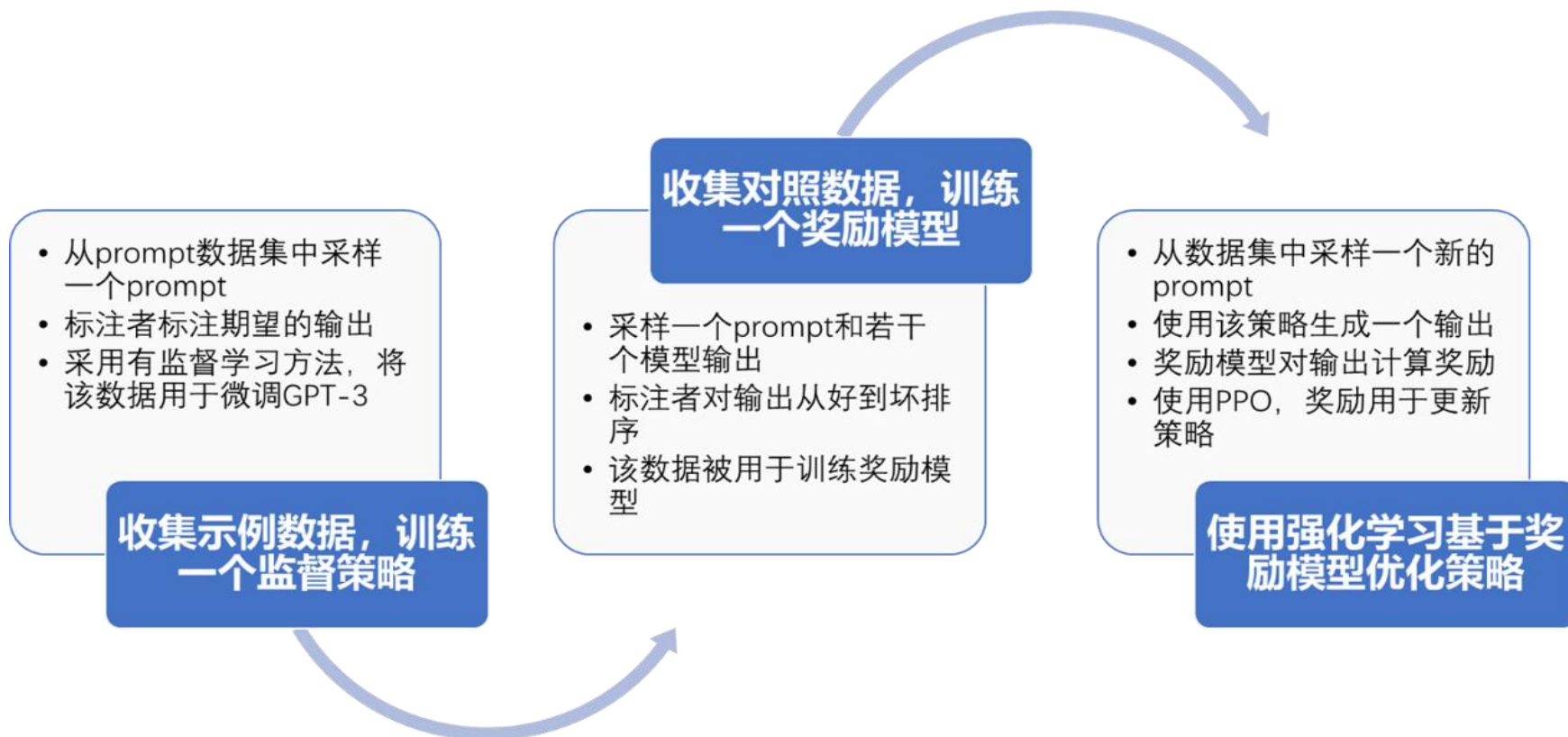
1	Translate English to French	← 任务描述
2	Sea otter=>loutre de mer	← 样例 (example)
3	peppermint=>menthe poivree	
4	Cheese=>	← 提示 (prompt)

InstructGPT

- ❑ 在之前的GPT系列模型中，给定任务的一些示例作为输入，GPT系列模型可以通过“提示（Prompt）”来执行自然语言处理领域的相关任务。然而仅仅依靠这些示例，模型通常会产生一些负面的输出，比如产生与原文事实不符的、或者不遵循用户指令、或者有偏见的文本内容。
- ❑ 具体来说，研究人员尝试使用基于人类反馈的强化学习方法来微调GPT-3模型，即以人类的偏好作为奖励信号来微调GPT模型。为了获得人类反馈，OpenAI聘请人员进行数据标记，评价GPT模型生成的文本。在初步获得人类反馈之后，使用这部分的数据进行GPT系列模型的有监督训练。

InstructGPT

□ InstructGPT模型工作流程图



InstructGPT

□ InstructGPT首先使用有监督学习在标签上微调GPT3，然后训练一个奖励模型来获取提示、响应和对应的奖励。在这个过程中的奖励模型（Reward Model）是一个有监督的微调模型，输入是Prompt和生成的答案内容，输出一个标量作为打分结果，奖励模型的损失函数如下：

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x, y_w, y_l) \sim D} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))]$$

其中， K 是模型生成的答案的个数， $r_\theta(x, y)$ 是提示 x 和生成 y 对应的奖励标量输出， θ 是参数， y_w 表示 (y_l, y_w) 模型所生成的序列中的偏好输出。**OpenAI**聘请的标注人员对 K 个模型生成的答案进行排序，目标函数的意义是使得奖励模型与人工标注的答案相对顺序一致。

InstructGPT

- 接下来，使用强化学习并利用奖励模型在环境中微调GPT模型，给定提示和响应，它产生由奖励模型确定的奖励得分，在强化学习的训练中，InstructGPT尝试最大化以下组合目标函数：

$$\begin{aligned} obj(\theta) \\ = E_{(x,y) \sim D_{\pi_{\theta}^{RL}}} \left[r_{\theta}(x,y) - \beta \log \left(\frac{\pi_{\theta}^{RL}(y|x)}{\pi^{SFT}(y|x)} \right) \right] + \gamma E_{(x) \sim D_{pretrain}} \left[\log \left(\pi_{\theta}^{RL}(x) \right) \right] \end{aligned}$$

- 其中， π_{θ}^{RL} 是强化学习策略， π^{SFT} 是有监督训练的模型， $D_{pretrain}$ 是数据的预训练分布。 β 是KL（KL散度距离）奖励系数， γ 是预训练损失系数。

ChatGPT

- ❑ ChatGPT是目前影响最大的GPT模型，它在人机对话领域超过了以往任何一个对话模型，取得了令人振奋的成绩。
- ❑ ChatGPT的工作模式与InstructGPT大致相似，只是使用的数据集有些区别。OpenAI目前尚未公布ChatGPT的详细技术细节，官方发布的博客中的大致描述是ChatGPT使用了对话精调的方式，即多轮Prompt和上下文，在训练InstructGPT模型中用到的数据集也被转换成对话的形式合并使用。
- ❑ 基于反馈的强化学习方法仍然被使用。



4

BERT系列模型

BERT简介

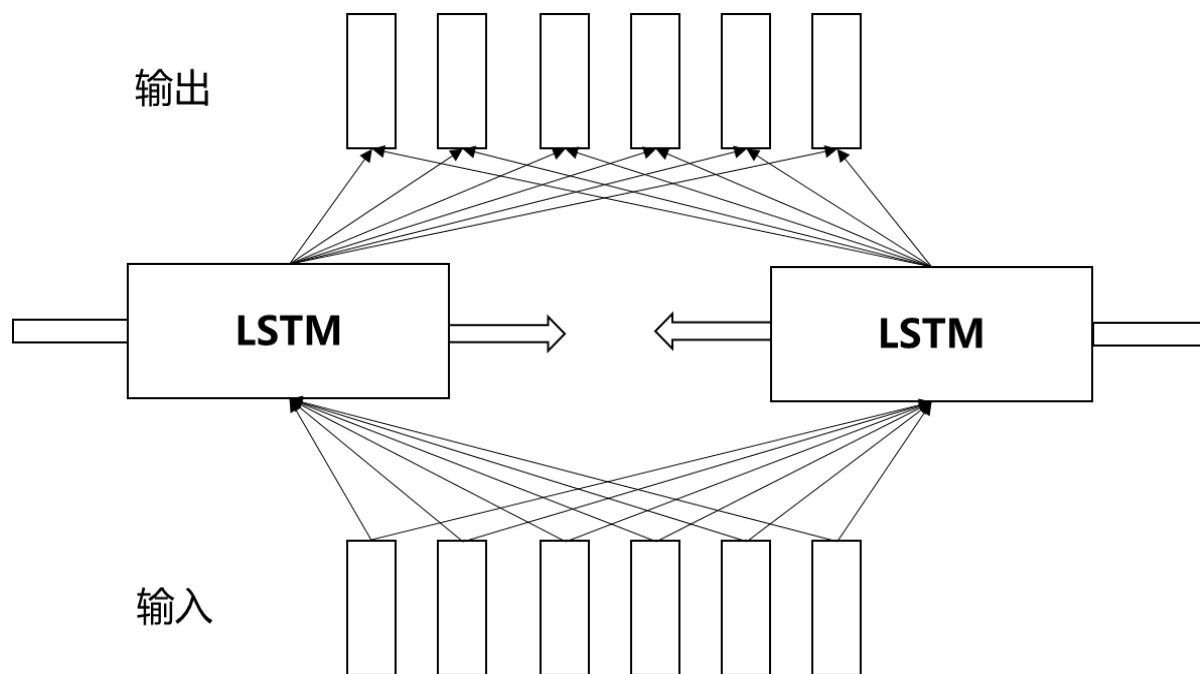
- ❑ 基于Transformer的双向编码器表示模型（Bidirectional Encoder Representations from Transformers, BERT）是一种**基于Transformer的纯Encoder模型**。
- ❑ 2018年由Google公司提出，广泛**应用于自然语言处理领域的各种任务**，比如句子分类、句子相似度计算、自然语言推理、命名实体识别、基于机器阅读理解的问答等。
- ❑ BERT**采用双向Transformer编码器作为基础结构**，它能够同时利用上下文信息和句子内部信息，提升模型的表达能力。
- ❑ BERT通过在大规模语料库上进行无监督的预训练，学习到通用的语言知识和上下文信息，然后在特定任务上进行微调，以适应不同的应用场景。

BERT简介

- ❑ 将预训练模型应用于下游任务有两种策略，分别是**基于特征的策略**和**基于微调的策略**。
- ❑ ELMO使用的是一种基于特征的方法，GPT系列模型使用的是一种微调的方法。这两种方法可以通过微调所有的预训练参数对下游任务进行训练，且它们都采用单向语言模型来学习通用的语言表示。

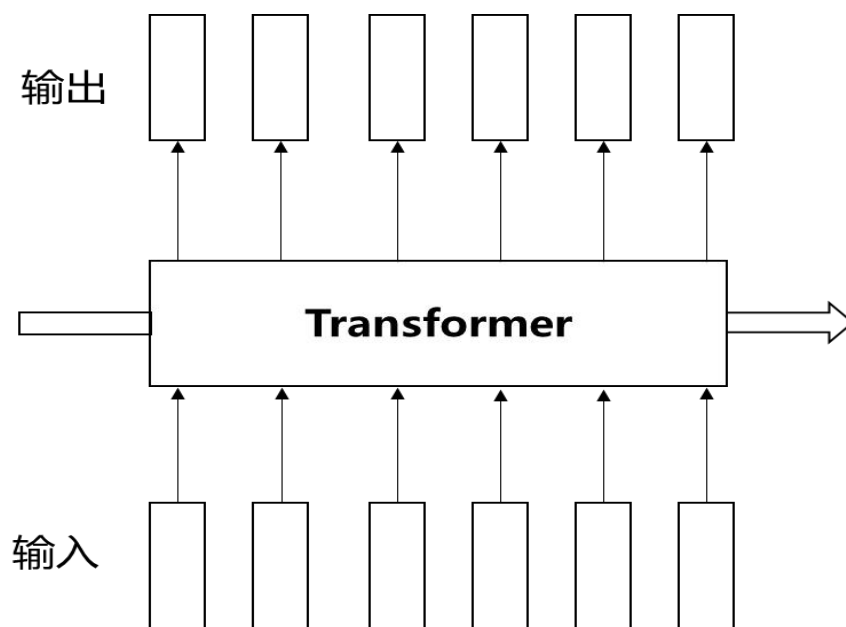
与其他大规模预训练模型的区别

- ELMO使用的是一种基于特征的方法应用到下游任务，使用自左向右和自右向左的两个LSTM网络作为编码器，分别以 $P(w_i|w_1, \dots, w_{i-1})$ 和 $P(w_i|w_{i+1}, \dots, w_n)$ 作为目标函数独立训练，将训练得到的特征向量以拼接的形式实现双向编码。



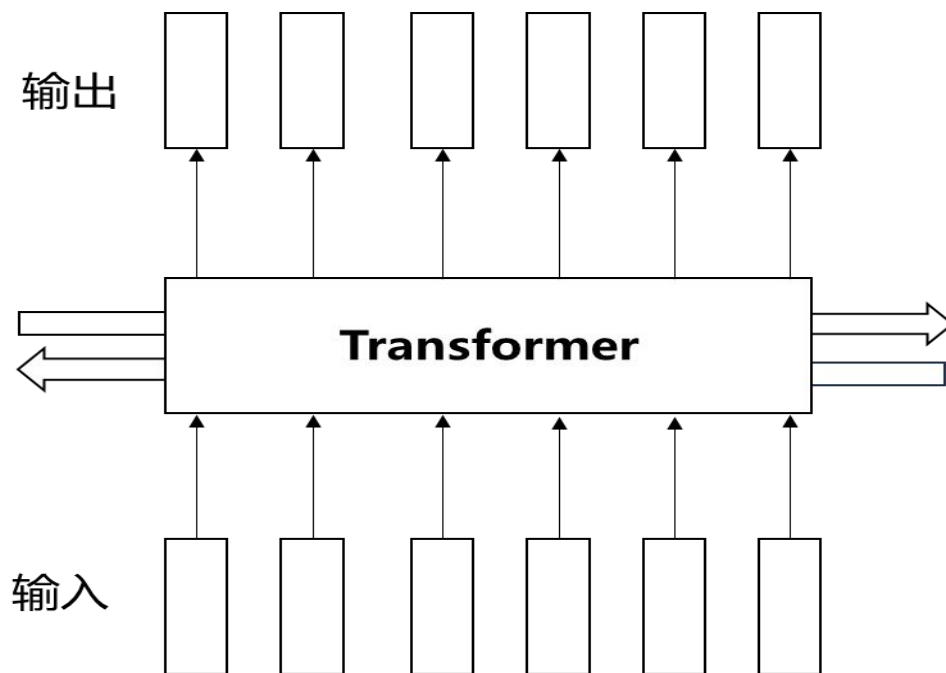
与其他大规模预训练模型的区别

- GPT系列模型使用一种微调的方法应用到下游任务，在Transformer解码器的基础上进行构建，以 $P(w_i|w_1, \dots, w_{i-1})$ 为目标进行训练，也是一个单向编码模型。



与其他大规模预训练模型的区别

- ❑ BERT的设计者认为单向的语言模型只能关注之前的单词，在对于诸如问答类任务时，效果可能不是最优的，能够从两个方向合并上下文信息是必要的。
- ❑ BERT使用 $P(w_i | w_1, \dots, w_{i-1}, w_{i+1}, \dots, w_n)$ 为目标函数进行训练，它是一个双向编码模型。



BERT的参数

- BERT模型基于Transformer模型的编码器构建，实现上与Transformer的细节几乎相同。首次提出的文献中定义Transformer的块数为 L ，隐藏层状态大小为 H ，自注意力机制的头数为 A 。
- 在Base模式的BERT中， $L=12$ ， $H=768$ ， $A=16$ ；在Large模式的BERT中， $L=24$ ， $H=1024$ ， $A=16$ 。

BERT的输入表示

- ❑ BERT将输入文本标记成Tokens，这些Tokens可以是单词、子词或者字符，然后将每个标记映射到其在词汇表中的唯一整数ID。BERT使用WordPiece算法将单词或者子词拆分成更小的单元
- ❑ 接下来，每个序列的第一个标记始终是一个特殊的分类标记[CLS]，与该标记对应的最终隐藏层状态被用作分类任务的聚合序列表示。一个句子对被打包成一个序列。
- ❑ 在句子之间增加[SEP]标记，为每个句子的每个标记增加段向量，指明其属于哪一个句子。

BERT的输入表示

输入单词	[cls]	my	dog	is	cute	[SEP]	he	likes	play	##ing	my
词向量	$E_{[cls]}$	E_{my}	E_{dog}	E_{is}	E_{cute}	$E_{[SEP]}$	E_{he}	E_{likes}	E_{play}	$E_{##ing}$	$E_{[SEP]}$
	+	+	+	+	+	+	+	+	+	+	+
段向量	E_A	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B
	+	+	+	+	+	+	+	+	+	+	+
位置向量	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}

BERT的训练

- ❑ BERT同样采用两阶段训练方法，分别是在大规模无标注语料上的预训练和在特定任务有标签数据上进行的微调。
- ❑ 在预训练阶段，BERT使用了掩码语言模型（Masked Language Model, MLM）和下一句预测（Next Sentence Prediction, NSP）两种方法进行预训练。

BERT的训练

- MLM是在输入文本中随机选择一部分单词，将它们替换成特殊的 [MASK] 标记，然后要求模型预测输入句子中被随机掩盖的单词。
- 值得注意的是，考虑到在测试阶段 [MASK] 标记并不会出现，这会导致预训练和微调之间存在一些不匹配的问题。
- 为了缓解这种情况，在实际的训练过程中，并不总是使用 [MASK] 替换需要被掩码的单词，而是选择在一个小概率的情况下使用随机标记替换需要被掩码的单词，如以15%概率选择要被掩码的单词，同时随机标记有三种情况：
[MASK]（占80%）、其他任意单词（占10%）和保持原单词不变（占10%）。

BERT的训练

- ❑ 在许多下游任务中，比如问答、自然语言推理和文本蕴含任务都需要理解两个句子之间的关系。
- ❑ 为使得BERT能够理解句子之间的关系，NSP任务要求模型判断两个句子之间是否是连续的，这个任务可以帮助语言模型学习到句子之间的关系和逻辑关联。
- ❑ BERT在大规模Wikipedia和BookCorpus语料库上进行预训练，包含340亿个单词。对于英语百科数据，只使用其中的文本片段，而忽略列表、表格和标题，使用文档级语料库而不是句子级别的语料库。

BERT的训练

- 在微调阶段，可以将BERT应用于各种具体的自然语言处理任务，对于文本分类任务，可以将BERT的输出向量输入到一个全连接层进行分类；对于命名实体识别任务，可以将BERT的输出序列输入到一个序列标注模型中进行命名实体识别。

BERT的变种

- BERT也衍生出了诸多变种模型，比如RoBERTa, ALBERT、BART和ERNIE等，不断提升了各项自然语言处理任务的性能。
- BERT中的训练方法存在一些缺点：
 - 首先，MLM直接对单个Token进行随机掩码，丢失了短语和实体信息，这一点对中文尤其明显，百度公司提出的ERNIE模型对其进行了改进；
 - 第二，MLM仅预测被掩码的单词，其他的单词没有参与到预测当中；
 - 第三，NSP任务可以学到序列层的信息，但是这仅仅是一个二分类问题，而且负样本构造过于简单，导致模型不能进行充分的训练，ALBERT和RoBERTa对其进行了相应的改进。

BERT的变种

□ RoBERTa对BERT进行了改进

- 首先，原始的BERT模型使用的是静态掩码，而RoBERTa使用的是动态掩码，在预训练的不同时刻，重新挑选15%的单词进行掩码。
- 其次，原始BERT模型在预训练的过程中，训练NSP的样本要么是从同一个文档中连续采样，要么来自不同的文档，这种设置过于简单，RoBERTa尝试了多种变形，发现在去除NSP任务的时候，模型的性能能够获得提升。
- 再次，BERT采用的是字符级别的编码，词汇表大小是30k，RoBERTa在训练中采用字节级别的编码，词汇表大小扩展到了50k。
- 最后，RoBERTa在训练中增加了批处理的大小。

BERT的变种

- ALBERT模型对BERT模型进行了参数方面的优化，具体来说，ALBERT通过两种参数缩减技术进行建模：
 - 第一种技术是分解嵌入参数，将大词汇嵌入矩阵分解成两个小矩阵，然后将隐藏层的大小与词向量的大小分开；
 - 第二种技术是跨层共享，BERT模型使用了12层不同的编码器，参数并没有共享，ALBERT模型将这12层的编码器的参数实现了共享。
- 同时，提出用于句子顺序预测的自监督损失（Sentence-order Prediction, SOP）任务替代NSP任务，SOP任务关注句子之间的连贯性，将NSP任务中的负样本换成了同一篇文章中的两个逆序句子，实验证明SOP任务带来的提升比NSP任务更好。

BERT的变种

- ❑ 绝大多数大规模预训练模型仅仅通过上下文预测缺失的单词对文本表示进行建模，这些工作不考虑句子中的先验知识。ERNIE通过知识掩码策略来解决这一问题，除了最基本的单词标记掩码策略之外，ERNIE增加了两种与知识相关的策略，分别是短语策略和实体策略。
 - 它将短语（如a series of）或者实体（J. K. Rowling）作为一个知识单元，在预训练期间，对这些知识单元进行掩码，这样的训练过程能够使大模型实现与外部知识的结合。
- ❑ 在模型架构方面，ERNIE模型同样采用多层Transformer模型作为基础编码器，对于中文语料库，设计者在CJK Unicode范围内的每个字符周围添加了空格，并使用Word Piece对中文句子进行标记。

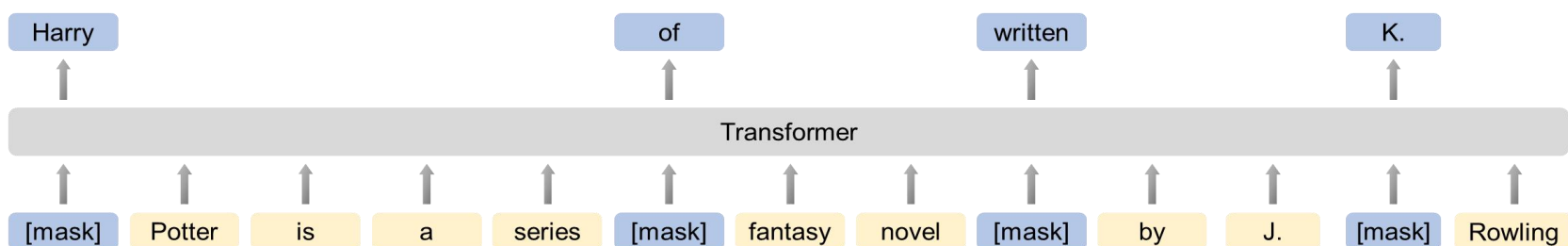
BERT的变种

- 此外，ERNIE提出了一种多阶段的知识掩码策略，将短语和实体级别的知识融合到语言表示中。
 - 第一个阶段是基本级别的掩码，它将句子视为最基本的语言单元，在训练中随机屏蔽15%的基本单元。
 - 第二个阶段是短语级别的掩码，短语是一组小的单词或者字符。
 - 第三个阶段是实体级别的掩码，包括人员、位置、产品等。
- ERNIE还引入了对话语言模型（Dialogue Language Model, DLM），DLM任务能够让ERNIE学习到对话中的隐含关系，进一步提升模型的语义表达能力。

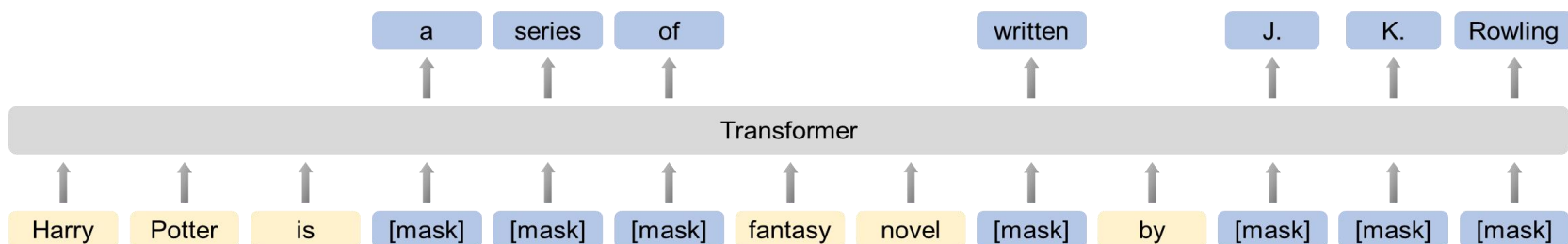
BERT的变种

□ BERT与ERNIE的结构比较

BERT



ERNIE





5

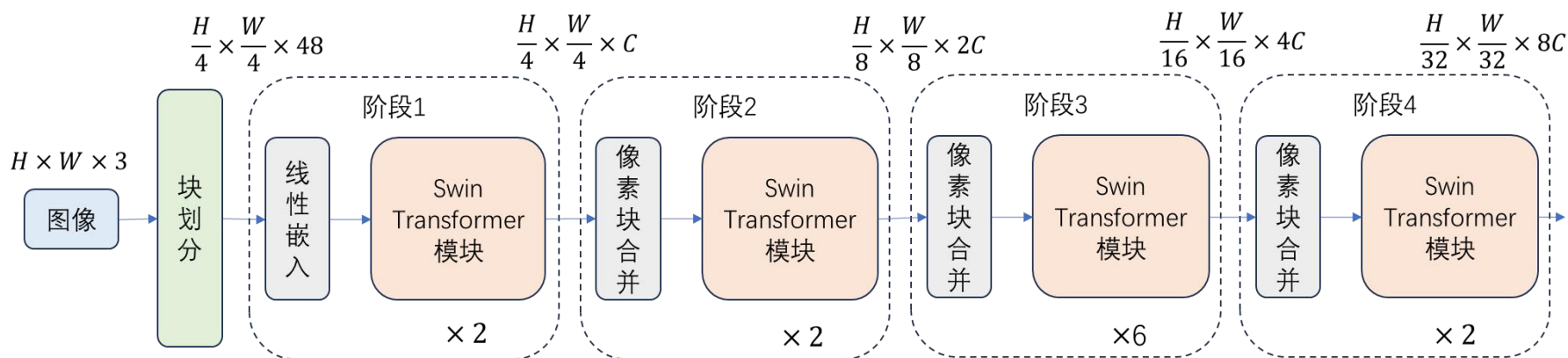
Swin Transformer

Swin Transformer的提出

- ❑ 把Transformer从自然语言处理领域迁移到计算机视觉领域主要面临两个挑战：
 - 不同视觉主体存在很大尺寸差异；
 - 图像像素比起自然语言文本而言有很高的分辨率。
- ❑ 针对这些挑战，Ze Liu等人提出了Swin Transformer，它是一种基于滑动窗口机制、采用层级设计的视觉Transformer，可以作为所有计算机视觉任务的骨干网络。
- ❑ Swin Transformer在ImageNet-1K数据集上取得了87.3的Top-1准确率，在其他密集视觉任务，如目标检测、语义分割等任务上，Swin Transformer也有非常优秀的表现。

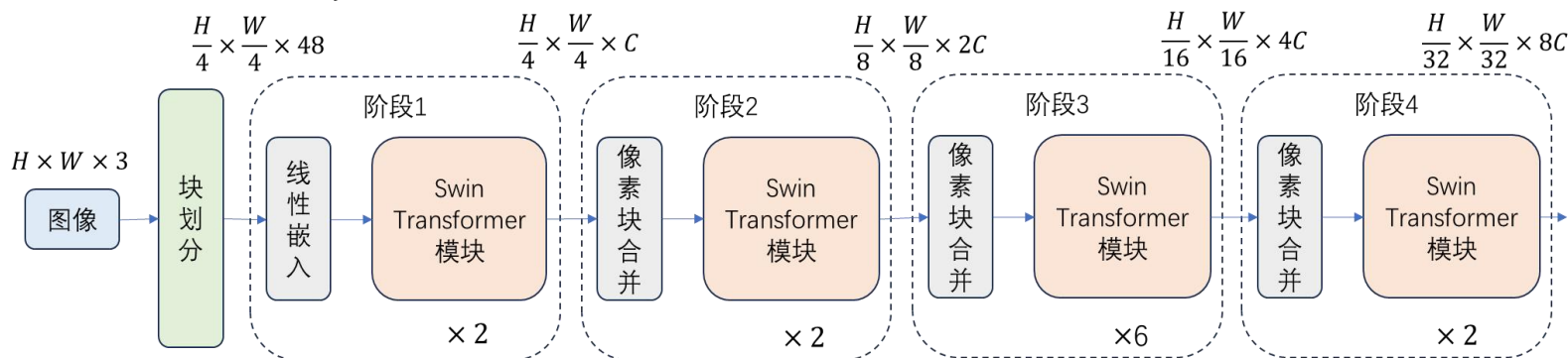
Swin Transformer的结构

□ Swin Transformer的结构采用层级化设计，一共分为4个阶段。



Swin Transformer的结构

- ❑ Swin Transformer首先将图像输入分成像素块，像素块大小为 4×4 。以图像大小 $224 \times 224 \times 3$ 为例，经过第一层后数据维度变为 $56 \times 56 \times (4 \times 4 \times 3)$ ，之后送入阶段1进行处理。
- ❑ 在阶段1中，数据从线性嵌入层出来之后，维度大小变为 3136×96 ，这对于Transformer块来说过大，因此需采用窗口内自注意力计算减小计算复杂度。
- ❑ 从阶段2开始，之后的每一阶段都是一个像素块合并层和Swin Transformer块，其中像素块合并层类似于卷积神经网络中的池化层，经过像素块合并层之后，数据的宽和高为原先的 $1/2$ ，通道数为原先的2倍。

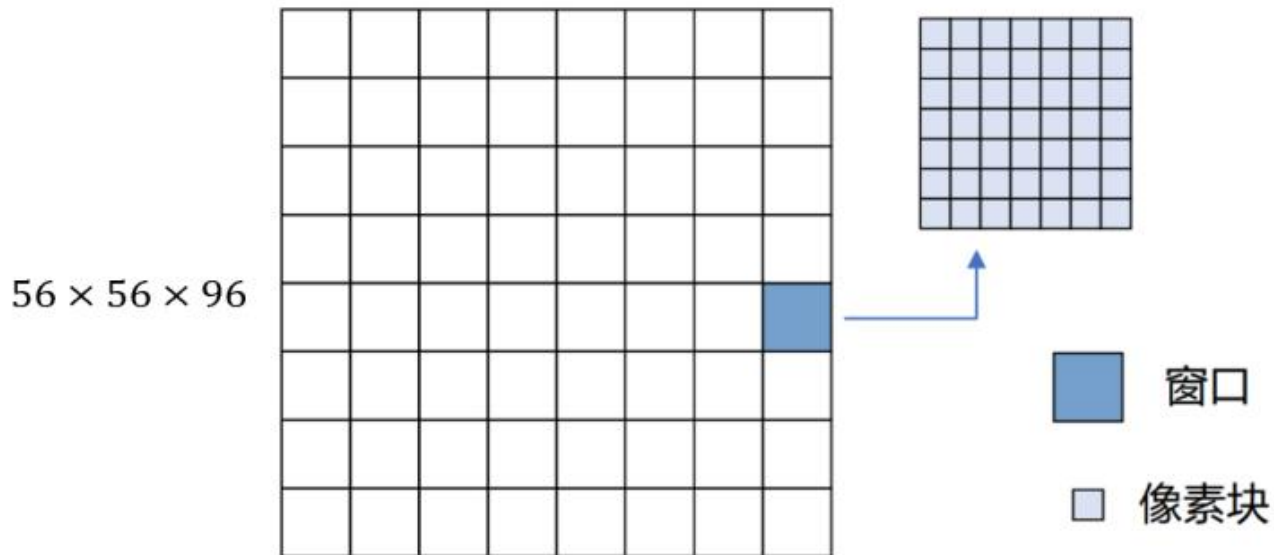


Swin Transformer的结构

□ 窗口内自注意力计算

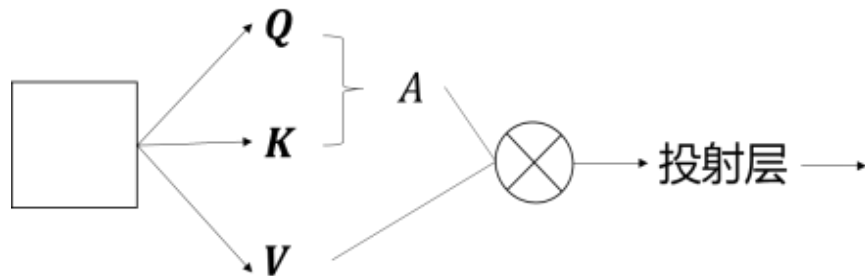
- 经过阶段1的线性嵌入层之后，一个 $224 \times 224 \times 3$ 的图像输入维度变成 3136×96 ，3136对于Transformer块来说过大，为了减小计算复杂度，Swin Transformer提出在窗口内进行自注意力计算。
- 下图展示的是一个 $56 \times 56 \times 96$ 的输入在窗口内计算时窗口与像素块关系说明。首先，默认窗口内有 7×7 的像素块，因此就有 $(56/7) \times (56/7) = 8 \times 8 = 64$ 个窗口。在窗口内计算自注意力让计算复杂度与图像大小呈线性相关，大大减小了计算复杂度。

Swin Transformer的结构



- 具体来说，自注意力计算流程如下图所示。向量与三个系数矩阵相乘后分别得到Q、K和V。然后Q与K相乘得到自注意力A，A再与V相乘，然后经过投射层得到最终结果。

Swin Transformer的结构



- 普通的自注意力计算复杂度和窗口内自注意力计算复杂度如公式下所示，其中M为窗口大小，在这里默认为7（即窗口里有7×7个像素块）。

$$\Omega(MSA) = 4hwC^2 + 2(hw)^2C$$

$$\Omega(W-MSA) = 4hwC^2 + 2M^2C$$

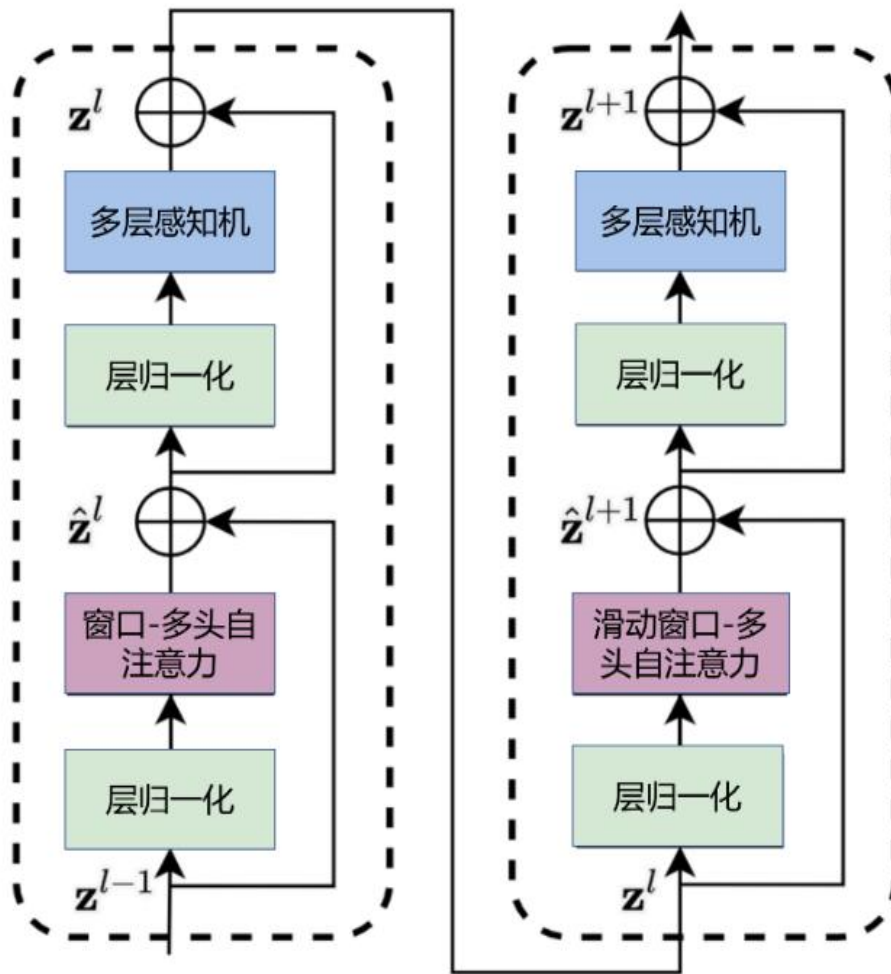
Swin Transformer的结构

□ Swin Transformer模块

- Swin Transformer是通过将Transformer模块中的多头自注意力（Multi-head Self-attention, MSA）模块替换为基于滑动窗口的模块来构建的，其它层保持不变。
- 一个Swin Transformer模块由一个基于滑动窗口的MSA模块组成，其后是一个2层的多层感知机（Multi-Layer Perceptron, MLP），GELU非线性介于两者之间。
- 在每个MSA模块和每个MLP之前应用一个层归一化（Layer Normalization），在每个模块之后应用一个残差连接。

Swin Transformer的结构

□ Swin Transformer模块



Swin Transformer的结构

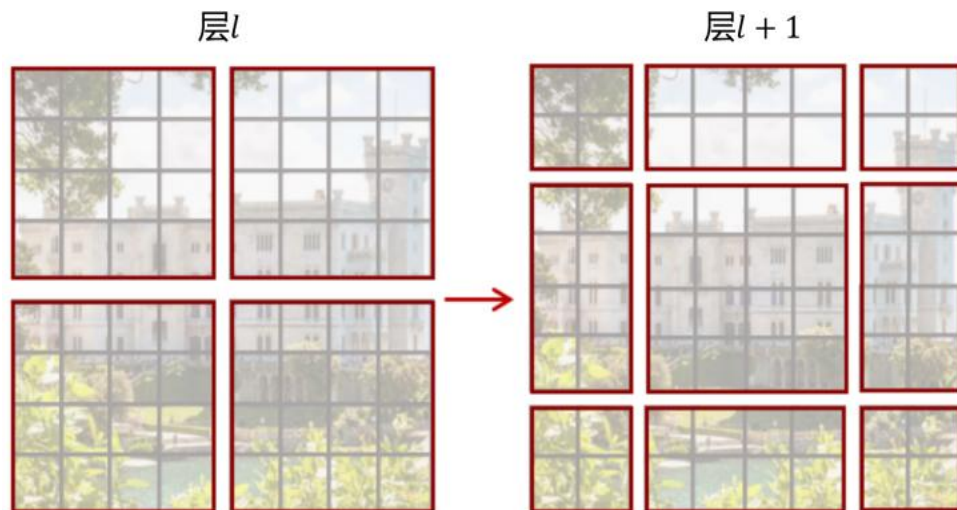
□ 像素块合并层

- 像素块合并层类似卷积神经网络中的池化层，对特征进行下采样。这个过程会让空间维度变小，但是通道数会增加。
- 比如原先 $H \times W \times C$ 的输入，经过下采样后维度变为 $H/2 \times W/2 \times 4C$ ，这时采用一个 1×1 的卷积后，维度就会变为 $H/2 \times W/2 \times 2C$ 。

Swin Transformer的窗口滑动机制

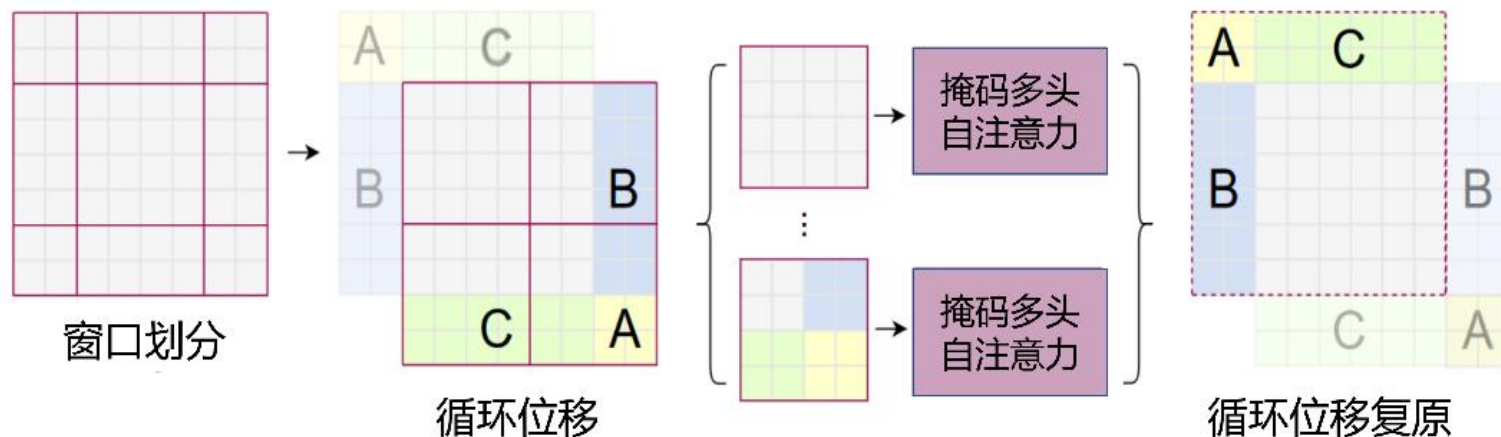
□ Swin Transformer使用窗口内自注意力计算，虽然可以大大减小计算复杂度，但是也让不同像素块缺少与其他像素块之间的联系。为了增加感受野，变相达到全局建模的能力，提出了滑动窗口机制。

- 如下图所示，红色正方形围住的部分是窗口，每个窗口内灰色的正方形为像素块。每次滑动时，窗口都向右下方向移动2个像素块。
- 这样，一些像素块就从原先的窗口移动到了新的窗口，如此再进行窗口内自注意力计算时，就得到了跨窗口的计算，变相实现了全局建模。



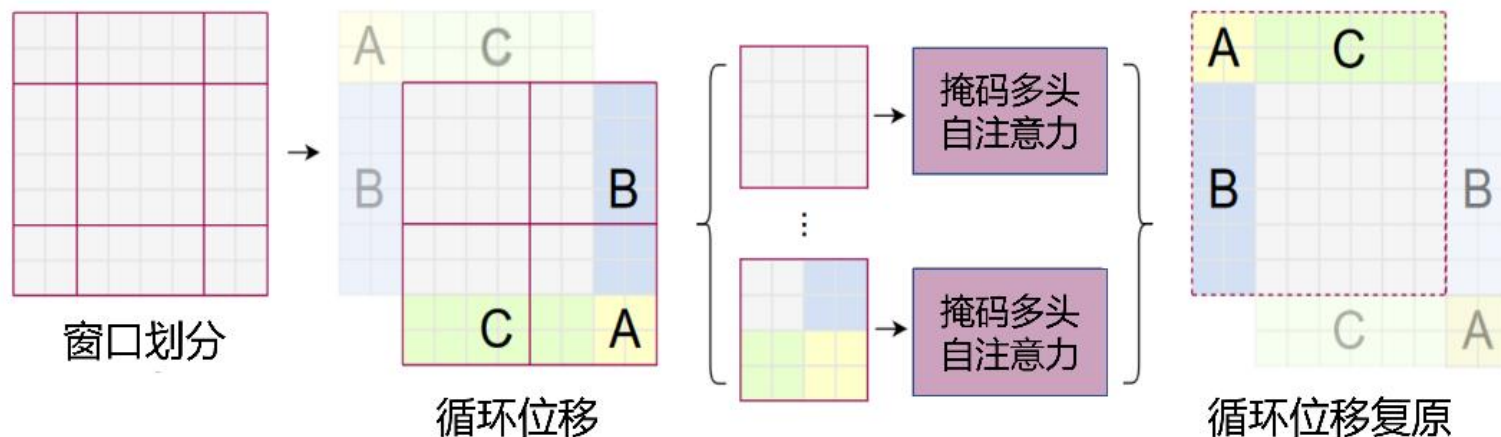
Swin Transformer的窗口滑动机制

- 但是，这样的滑动机制也造成了问题。如上图中， l 层有4个窗口， $(l+1)$ 层就变成了大小不一的9个窗口，对计算造成不便，也使计算复杂度增加。
- Swin Transformer使用了一种巧妙的循环位移和掩码方式，解决滑动窗口带来的不便。如下图所示，在窗口正常滑动之后，把其它非正常大小的窗口填充到各个部分，如左上角窗口A被移动到右下角，上方的窗口C被填充到下方，左边的窗口B被填充到右方。



Swin Transformer的窗口滑动机制

- ❑ 但是在这样得到的新窗口内可能存在两个本来相距很远的像素块，它们本来不应该放在一起计算，因此使用掩码多头自注意力解决。
- ❑ 在计算完毕后，再将其复原，否则该图就不断往右下方向循环下去了。



谢谢！



计算机科学与技术学院

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY